



AWS Academy Cloud Architecting (LA)

Module 13 Student Guide

Versión 2.0.13

200-ACACAD-20-LA-SG

© 2023, Amazon Web Services, Inc. o sus empresas afiliadas. Todos los derechos reservados.

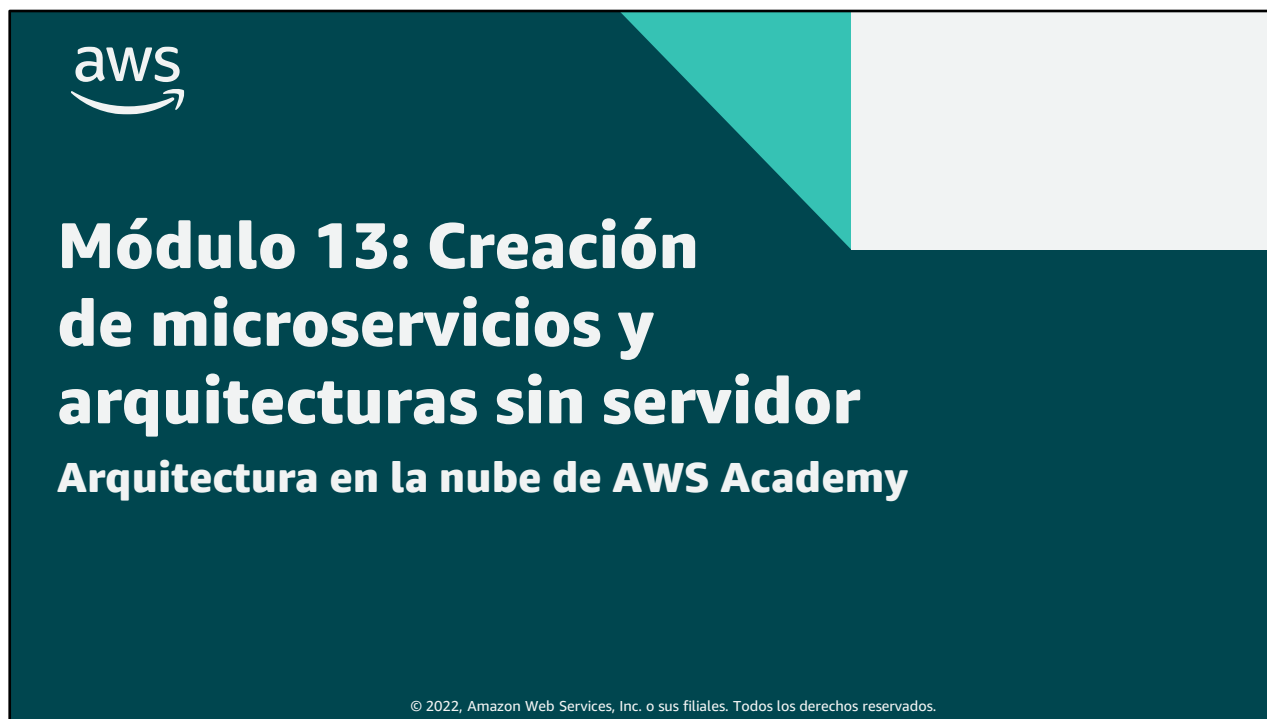
Este contenido no puede reproducirse ni redistribuirse, total ni parcialmente, sin el permiso previo por escrito de Amazon Web Services, Inc. Queda prohibida la copia, el préstamo o la venta de carácter comercial.

Todas las marcas comerciales pertenecen a sus propietarios.

Contenido

Módulo 13: Creación de microservicios y arquitecturas sin servidor

4



Le damos la bienvenida al módulo 13: Creación de microservicios y arquitecturas sin servidor.

Información general sobre el módulo

Secciones

1. Necesidad de arquitectura
2. Presentación de los microservicios
3. Creación de aplicaciones de microservicios con los servicios de contenedores de AWS
4. Presentación de las arquitecturas sin servidor
5. Creación de arquitecturas sin servidor con AWS Lambda
6. Ampliación de las arquitecturas sin servidor con Amazon API Gateway
7. Organización de microservicios con AWS Step Functions



Demostraciones

- Creación de una función de AWS Lambda
- Uso de AWS Lambda con Amazon S3

Laboratorios

- (Opcional) laboratorio guiado 1: División de una aplicación monolítica Node.js en microservicios
- Laboratorio guiado 2: Implementación de una arquitectura sin servidor en AWS
- Laboratorio de desafíos: Implementación de una arquitectura sin servidor para la cafetería



Evaluación de conocimientos

© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

2

Este módulo incluye las siguientes secciones:

Necesidad de arquitectura

1. Presentación de los microservicios
2. Creación de aplicaciones de microservicios con los servicios de contenedores de AWS
3. Presentación de las arquitecturas sin servidor
4. Creación de arquitecturas sin servidor con AWS Lambda
5. Ampliación de las arquitecturas sin servidor con Amazon API Gateway
6. Organización de microservicios con AWS Step Functions

Este módulo también incluye:

- Dos demostraciones de AWS Lambda
- Un laboratorio guiado opcional en el que se refactoriza una aplicación monolítica en microservicios
- Un laboratorio guiado en el que implementará una arquitectura sin servidor en AWS con Amazon S3, AWS Lambda, Amazon DynamoDB y Amazon SNS
- Un laboratorio de desafíos en el que utiliza AWS Lambda y Amazon Simple Notification Service (Amazon SNS) para generar y enviar un informe de ventas diario para la cafetería

Finalmente, se le pedirá que complete una evaluación de conocimientos que pondrá a prueba su comprensión de los conceptos clave que se abordaron en este módulo.

Objetivos del módulo

Tras completar este módulo, será capaz de hacer lo siguiente:

- Indicar las características de los microservicios
- Refactorizar una aplicación monolítica en microservicios y utilizar Amazon ECS para implementar los microservicios en contenedores
- Explicar la arquitectura sin servidor
- Implementar una arquitectura sin servidor con AWS Lambda
- Describir una arquitectura común para Amazon API Gateway
- Describir los tipos de flujos de trabajo que admite AWS Step Functions



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

3

Tras completar este módulo, serás capaz de hacer lo siguiente:

- Indicar las características de los microservicios
- Refactorizar una aplicación monolítica en microservicios y utilizar Amazon ECS para implementar los microservicios en contenedores
- Explicar la arquitectura sin servidor
- Implementar una arquitectura sin servidor con AWS Lambda
- Describir una arquitectura común para Amazon API Gateway
- Describir los tipos de flujos de trabajo que admite AWS Step Functions

Sección 1: Necesidad de arquitectura

Módulo 13: Creación de microservicios y arquitecturas sin servidor



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

Introducción a la sección 1: Necesidad de arquitectura.

Requisitos de la empresa de cafetería

La cafetería quiere recibir informes diarios por correo electrónico sobre todos los pedidos realizados en el sitio web. Necesitan esta información para poder anticiparse a la demanda y hornear el número correcto de postres en el futuro (reduciendo el desperdicio). También quieren identificar patrones en su empresa (análisis).



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

5

Frank y Martha quieren recibir informes diarios por correo electrónico sobre todos los pedidos realizados en el sitio web. Frank quiere anticiparse a la demanda y hornear el número correcto de postres en el futuro (reduciendo el desperdicio). Martha quiere identificar patrones en su empresa (análisis). Actualmente, Sofía ha configurado un trabajo cron en la instancia del servidor web que envía estos mensajes de correo electrónico con el informe diario de los pedidos a Frank y Martha. Sin embargo, el trabajo cron consume muchos recursos y reduce el rendimiento del servidor web.

Olivia aconseja a Sofía y a Nikhil separar los informes de las tareas que no sean críticas para la empresa. Sofía y Nikhil quieren desacoplar más la arquitectura y trasladar el trabajo cron a un entorno administrado y sin servidor, que escale bien y reduzca los costos.

Sección 2: Presentación de los microservicios

Módulo 13: Creación de microservicios y arquitecturas sin servidor



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

Introducción a la sección 2: Presentación de los microservicios.

¿Qué son los microservicios?

Aplicaciones compuestas por **servicios independientes**
que se comunican a través de **API bien definidas**

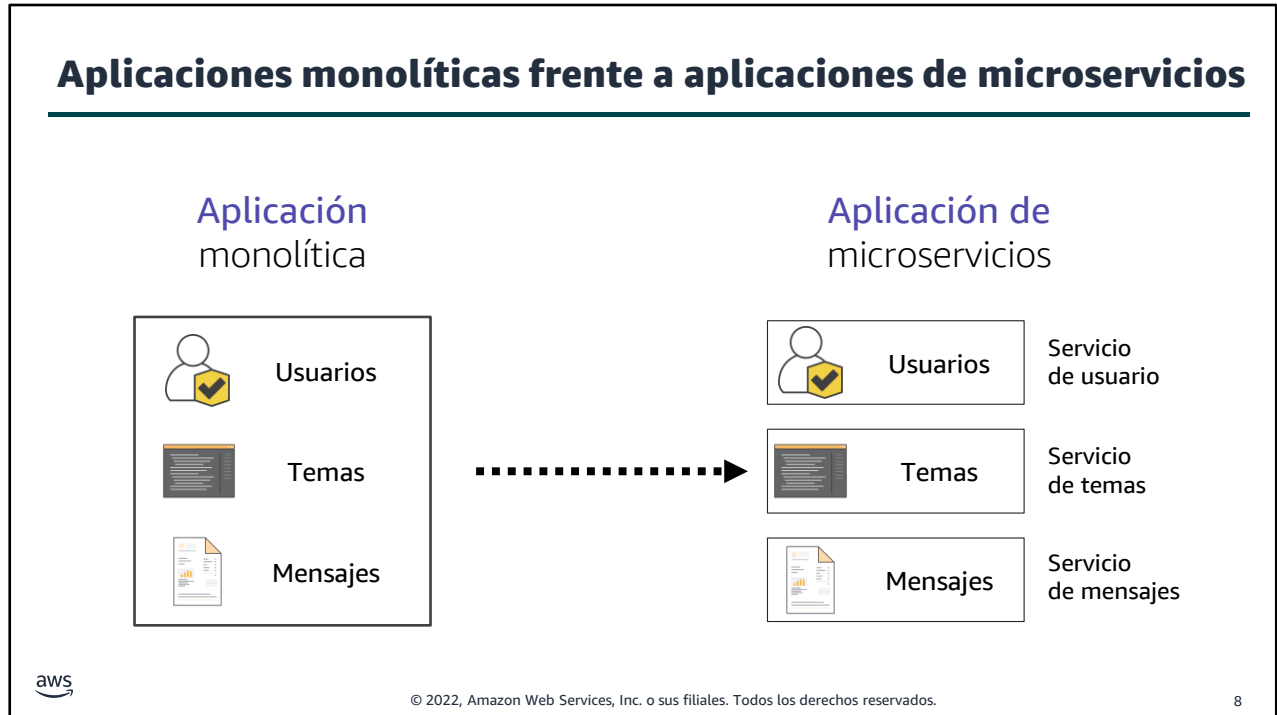


© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

7

Los microservicios son un enfoque de arquitectura y organización para el desarrollo de software en el que las aplicaciones se componen de servicios independientes que se comunican mediante interfaces de programación de aplicaciones (API) bien definidas. Este enfoque está diseñado para acelerar los ciclos de implementación.

El enfoque de microservicios fomenta la innovación y la propiedad, y mejora la capacidad de mantenimiento y escalabilidad de las aplicaciones de software.



Para comprender los beneficios de los microservicios, primero consideremos una aplicación monolítica.

En el ejemplo de la izquierda, los tres procesos (usuarios, temas y mensajes) de una aplicación monolítica de foro están estrechamente acoplados. Se ejecutan como un solo servicio. Si un proceso de una aplicación experimenta un pico de demanda, se debe escalar toda la arquitectura. Agregar o mejorar funciones se vuelve más complejo a medida que crece la base de código, lo que limita la experimentación y dificulta la implementación de nuevas ideas. La disponibilidad de las aplicaciones monolíticas también está en riesgo porque muchos procesos dependientes y con un acoplamiento ajustado aumentan el impacto de un error de un solo proceso.

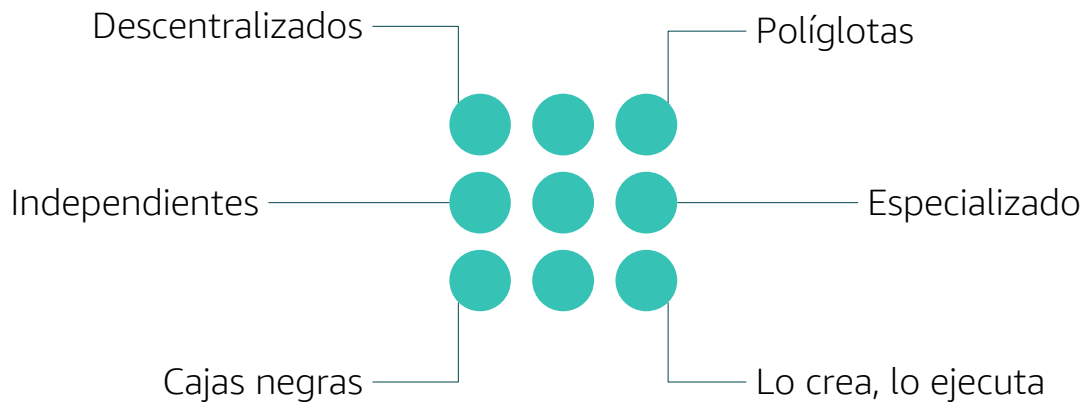
Supongamos ahora que la misma aplicación se ejecuta en una arquitectura de microservicios. Cada proceso de la aplicación se desarrolla como un componente independiente que se ejecuta como un servicio. Los servicios se comunican mediante operaciones ligeras de la API. Cada servicio realiza una única función que puede admitir múltiples aplicaciones. Debido a que los servicios se ejecutan de forma independiente, se pueden actualizar, implementar y escalar para satisfacer la demanda de funciones específicas de una aplicación.

Una arquitectura de microservicios proporciona una iteración, automatización y agilidad general mucho más rápidas. Iniciar rápido, encontrar errores rápido y recuperarse rápido.

Para ver la información general sobre los microservicios en AWS, consulte [¿Qué son los](#)

[microservicios?](#)

Características de los microservicios



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

9

Los microservicios comparten algunas características comunes:

- **Descentralizados:** las arquitecturas de microservicios son sistemas distribuidos con administración de datos descentralizada. No dependen de un esquema unificado en una base de datos central. Cada microservicio tiene su propia visión sobre los modelos de datos. Los microservicios también se desarrollan, implementan, administran y operan de forma descentralizada.
- **Independientes:** cada servicio de componente de una arquitectura de microservicios puede modificarse, actualizarse o reemplazarse de forma independiente sin que afecte la función de los demás servicios. Los servicios no necesitan compartir ninguno de sus códigos o implementaciones con otros servicios. Del mismo modo, los equipos responsables de los distintos microservicios pueden funcionar de forma independiente entre ellos.
- **Especializados:** cada servicio de componente está diseñado para un conjunto de funciones y se enfoca en resolver un dominio específico. Si el código de un servicio de componente determinado alcanza un cierto nivel de complejidad, el servicio puede dividirse en dos o más servicios.
- **Políglotas:** los microservicios no siguen un solo enfoque. Los equipos tienen la libertad de elegir la mejor herramienta para sus problemas específicos. Por lo tanto, las arquitecturas de microservicios adoptan un enfoque heterogéneo en los sistemas operativos, lenguajes de programación, almacenes de datos y herramientas. Este enfoque se denomina persistencia y programación políglota.
- **Cajas negras:** los servicios de componentes individuales se diseñan como cajas negras, lo que significa que los detalles de su complejidad están ocultos para otros componentes. Cualquier comunicación entre servicios ocurre a través de las API bien definidas para evitar dependencias implícitas y ocultas.
- **Lo crea, lo ejecuta:** DevOps es un principio organizativo clave para los microservicios, donde el equipo responsable de crear un servicio, también es responsable de operarlo y mantenerlo en

producción.

Conclusiones importantes de la Sección 2



- Las aplicaciones de microservicios se componen de **servicios independientes que se comunican a través de API bien definidas**
- Los microservicios comparten las siguientes características:
 - Descentralizados
 - Independientes
 - Especializado
 - Políglotas
 - Cajas negras
 - Lo crea, lo ejecuta

© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

10

Entre los puntos clave de esta sección del módulo, se incluyen los siguientes:

- Las aplicaciones de microservicios se componen de servicios independientes que se comunican a través de API bien definidas
- Los microservicios comparten las siguientes características:
 - Descentralizados: los microservicios se desarrollan, implementan, administran y operan de forma descentralizada
 - Independientes: cada servicio de componente en una arquitectura de microservicios se puede desarrollar, implementar, operar y escalar sin afectar el funcionamiento de otros servicios
 - Especializados: cada servicio está diseñado para un conjunto de funciones y se enfoca en resolver un problema específico
 - Políglotas: las arquitecturas de microservicios adoptan un enfoque heterogéneo en los sistemas operativos, lenguajes de programación, almacenes de datos y herramientas
 - Cajas negras: los detalles de la complejidad de los microservicios están ocultos para otros componentes
 - Lo crea, lo ejecuta: DevOps es un principio organizativo clave para los microservicios

Sección 3: Creación de aplicaciones de microservicios con los servicios de contenedores de AWS

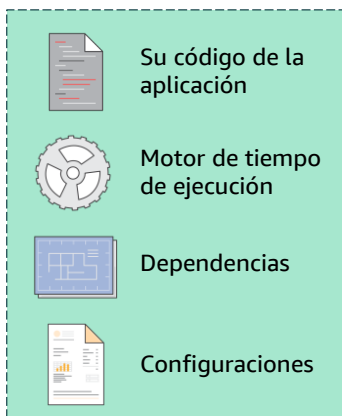
Módulo 13: Creación de microservicios y arquitecturas sin servidor



Introducción a la sección 3: Creación de aplicaciones de microservicios con los servicios de contenedores de AWS.

¿Qué es un contenedor?

Su contenedor



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

12

Cuando se construye una arquitectura de microservicios, se pueden utilizar contenedores para la potencia de procesamiento.

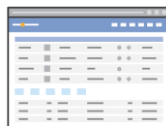
Los *contenedores* son un método de virtualización del sistema operativo que le permite ejecutar una aplicación y sus dependencias en procesos aislados de recursos. Un contenedor es un paquete de software ligero e independiente. Contiene todo lo que una aplicación de software necesita para ejecutarse, como el código de la aplicación, el motor de ejecución, las herramientas del sistema, las bibliotecas del sistema y las configuraciones.

Los contenedores pueden resolver el problema

Lograr que el software **funcione correctamente**
en distintos entornos de trabajo



Estación de trabajo
del desarrollador



Entorno
de producción



Entorno
de pruebas



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

13

Los contenedores pueden ayudar a garantizar que las aplicaciones se implementen de forma rápida, fiable y coherente, cualquiera que sea el entorno de implementación. Los contenedores también le ofrecen más control granular sobre los recursos, lo que mejora la eficiencia de su infraestructura.

Terminología de los contenedores



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

14

Un contenedor se crea a partir de una plantilla de solo lectura que se denomina *imagen*. Las imágenes se suelen crear a partir de un Dockerfile, que es un archivo de texto plano que especifica todos los componentes incluidos en el contenedor. Puede crear imágenes desde cero o utilizar imágenes creadas por otros y publicadas en un registro de contenedores público o privado.

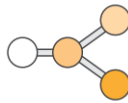
Una imagen de contenedor es una instantánea del sistema de archivos que está disponible para el contenedor. Por ejemplo, puede tener el sistema operativo Debian como una imagen de contenedor. Cuando ejecute este contenedor, tendrá disponible un sistema operativo Debian. También puede empaquetar todas sus dependencias de código en la imagen del contenedor y usarla como artefacto de código.

Las imágenes de los contenedores se almacenan en un *registro*. Puede descargar las imágenes del registro y ejecutarlas en su clúster. Los registros pueden existir dentro o fuera de su infraestructura de AWS.

Amazon ECS



Coordina la ejecución de los contenedores



Mantiene y escala la flota de instancias que ejecutan sus contenedores



Elimina la complejidad de crear la infraestructura



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

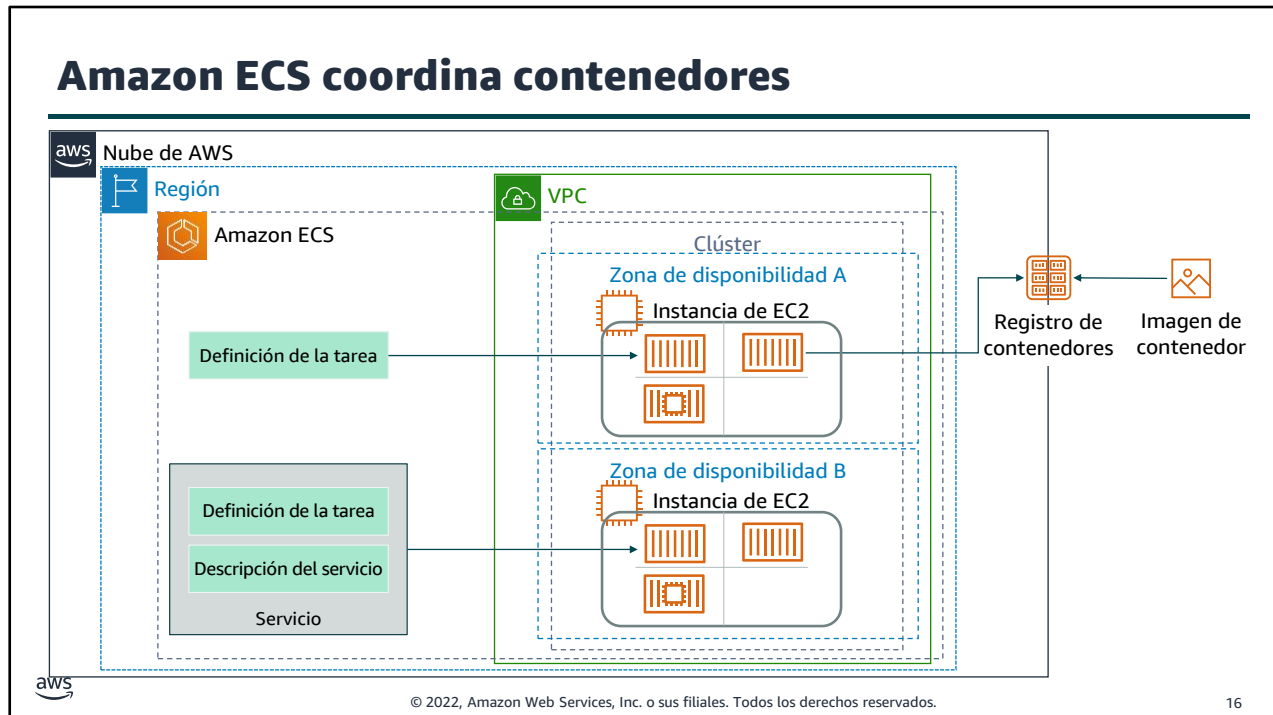
15

Puede ejecutar sus contenedores de Amazon Elastic Container Service (Amazon ECS). Amazon ECS es un servicio de administración de contenedores altamente escalable y de alto rendimiento. Admite contenedores de Docker y permite ejecutar aplicaciones fácilmente en un clúster administrado de instancias de Amazon Elastic Compute Cloud (Amazon EC2).

Amazon ECS es un servicio de clúster escalable para alojar contenedores que:

- Puede escalar hasta miles de contenedores de Docker en segundos
- Supervisa la implementación de contenedores
- Administra el estado del clúster que ejecuta los contenedores
- Programa contenedores con un programador integrado o de terceros (Apache Mesos o Blox)
- Se puede ampliar mediante API
- Se puede iniciar con los [tipos de lanzamiento](#) de AWS Fargate o Amazon EC2

Puede ejecutar clústeres de ECS a escala combinando instancias de spot con instancias bajo demanda y reservadas.



Amazon ECS es un servicio regional que simplifica la ejecución de contenedores de aplicaciones con alta disponibilidad en varias zonas de disponibilidad dentro de una región. Puede crear clústeres de ECS en una nube virtual privada (VPC) nueva o existente. Un clúster es un agrupamiento lógico de recursos.

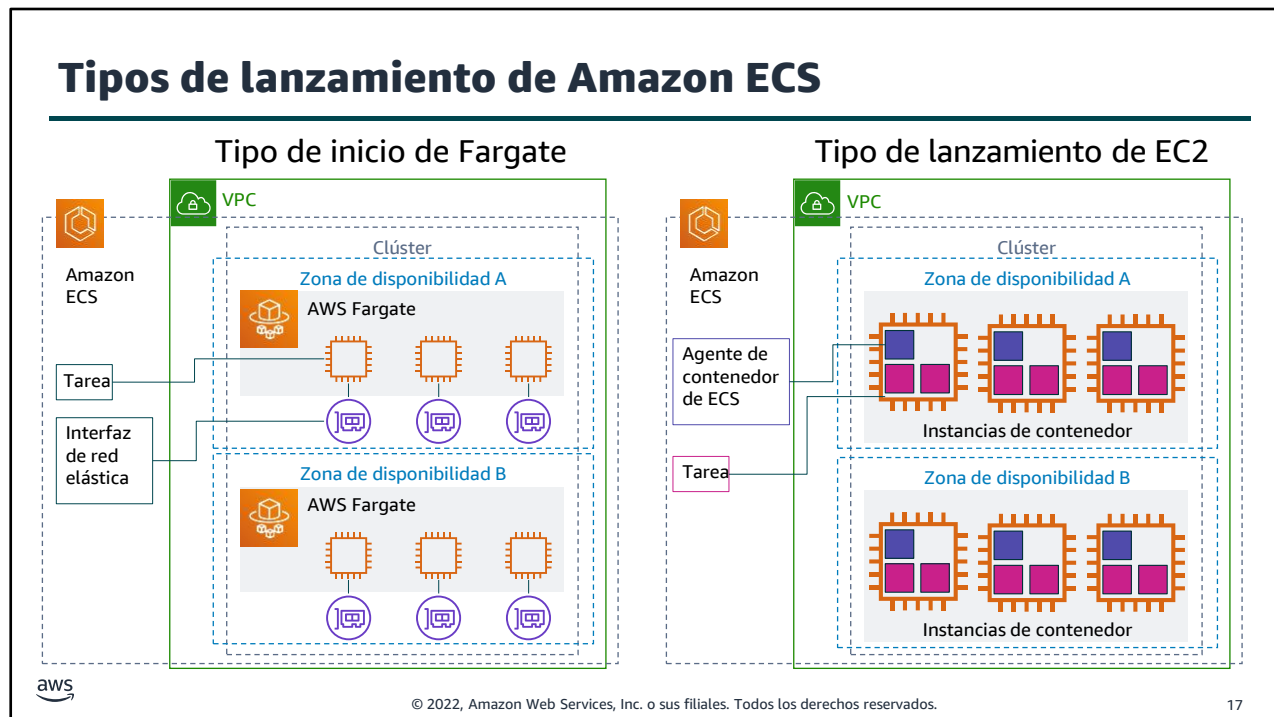
Una vez que un clúster está en funcionamiento, puede definir tareas y servicios que especifiquen qué imágenes de contenedores de Docker se ejecutarán en los clústeres.

Una definición de tarea es un archivo de texto en formato JavaScript Object Notation (JSON). Describe uno o más contenedores, hasta un máximo de 10, que forman la aplicación. Puede considerarlo como un esquema para su aplicación. Las definiciones de tareas especifican parámetros para su aplicación, por ejemplo, los contenedores y el tipo de inicio que debe utilizar. Otros parámetros incluyen qué puertos se deben abrir para su aplicación y qué volúmenes de datos se deben utilizar con los contenedores en la tarea.

Un servicio le permite determinar cuántas copias de la definición de tareas se ejecutarán y mantendrán en un clúster. Si lo desea, puede elegir utilizar un equilibrador de carga de Elastic Load Balancing para distribuir el tráfico entrante a los contenedores de su servicio. Amazon ECS mantiene ese número de tareas y coordina la programación de las tareas con el equilibrador de carga.

Después de crear una definición de tarea para su aplicación, puede especificar la cantidad de tareas que se ejecutarán en el clúster. Una tarea es la instancia de una definición de tareas dentro de un clúster. Cuando utiliza Amazon ECS para ejecutar tareas, estas se colocan en un clúster.

Amazon ECS descarga las imágenes de contenedor desde un registro específico y las ejecuta en el clúster.



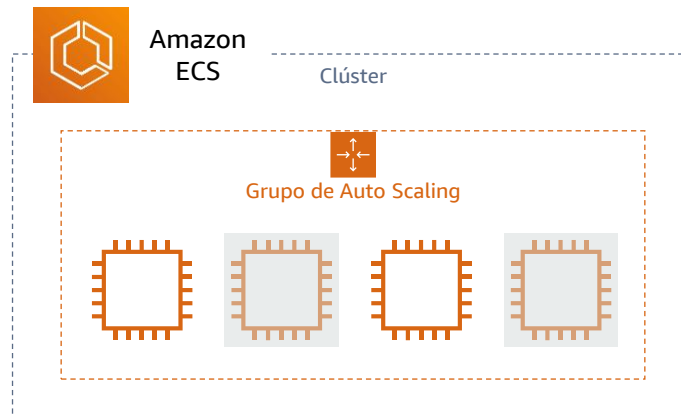
Amazon ECS ofrece dos tipos de lanzamiento para alojar sus aplicaciones en contenedores.

Puede utilizar el tipo de lanzamiento de Fargate para alojar su clúster en una infraestructura sin servidor administrada por Amazon ECS. Solo necesita empaquetar la aplicación en contenedores, especificar los requisitos de CPU y memoria, definir las políticas de redes y de AWS Identity and Access Management (AWS IAM) e iniciar la aplicación.

Por otro lado, si desea un mayor control, puede utilizar el tipo de lanzamiento de EC2 para alojar sus tareas en un clúster de instancias de contenedor de EC2 que *usted administre*. Una *instancia de contenedor* es una instancia de EC2 que ejecuta el *agente de contenedores* de Amazon ECS. Puede usar Amazon ECS para programar la ubicación de los contenedores en el clúster en función de sus necesidades de recursos, las políticas de aislamiento y los requisitos de disponibilidad. Para obtener información sobre las diferentes opciones de programación, consulte [Programar las tareas de Amazon ECS](#). Amazon ECS realiza un seguimiento de toda la CPU, la memoria y otros recursos del clúster. También encuentra el mejor servidor para que ejecute el contenedor en función de los requisitos de los recursos necesarios.

Para obtener más información sobre los tipos de lanzamiento de Fargate y EC2, consulte [Tipos de lanzamiento de Amazon ECS](#).

Escalado automático de clústeres de Amazon ECS



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

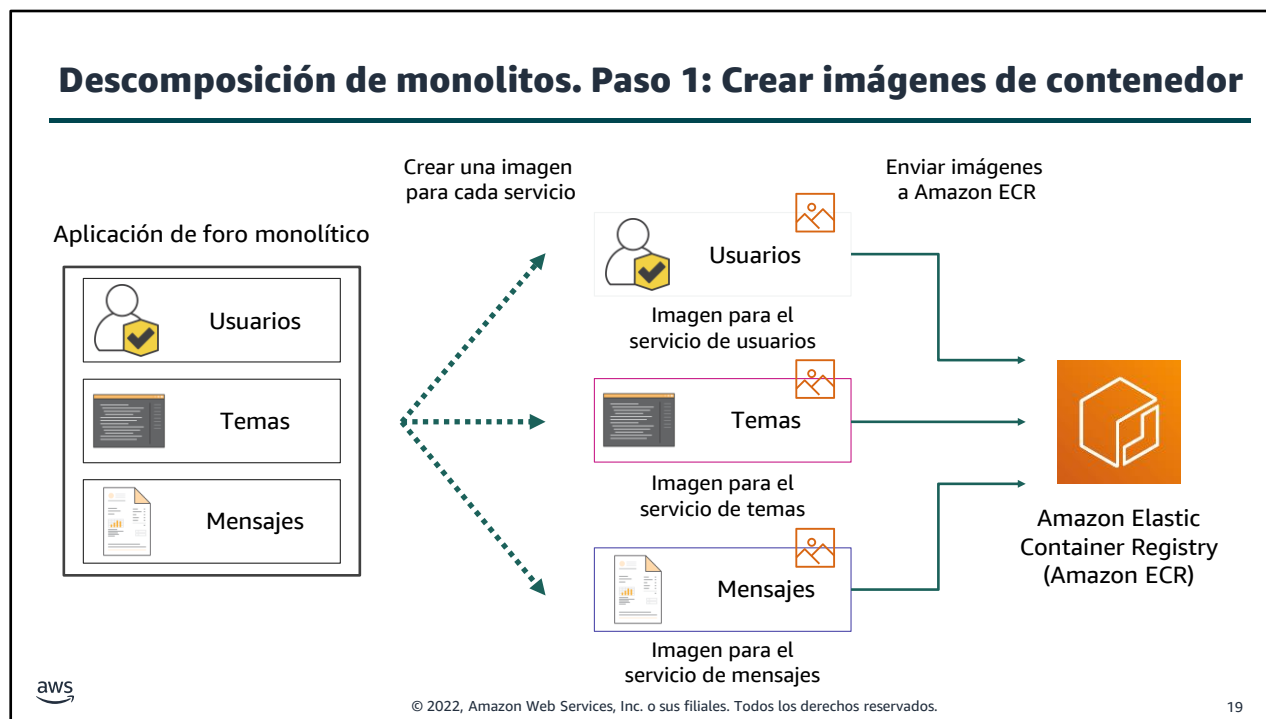
18

Puede [crear un grupo de Auto Scaling para un clúster de Amazon ECS](#). El grupo de Auto Scaling incluye instancias de contenedor que puede escalar horizontalmente (y reducir) mediante las alarmas de Amazon CloudWatch. Si configura su grupo de Auto Scaling para eliminar instancias de contenedor, se detendrán todas las tareas que se estén ejecutando en las instancias de contenedor eliminadas. Si sus tareas se ejecutan como parte de un servicio, Amazon ECS las reinicia en otra instancia si los recursos necesarios están disponibles. Algunos ejemplos de los recursos necesarios son la CPU, la memoria y los puertos. Sin embargo, las tareas iniciadas de forma manual no se reinician automáticamente.

También puede aprovechar el [escalado automático de clústeres de Amazon ECS](#), que le ofrece más control sobre cómo escalar las tareas en un clúster. Aumenta la velocidad y la fiabilidad del escalado de clústeres. Le ofrece control sobre la cantidad de capacidad adicional que se mantiene en su clúster y administra automáticamente la terminación de instancias al reducir horizontalmente.

Con el escalado automático del clúster, puede configurar Amazon ECS para que escale o reduzca horizontalmente su grupo de Auto Scaling de forma automática. El escalado automático de clústeres se basa en proveedores de capacidad, que vinculan su clúster de ECS a los grupos de Auto Scaling que desea utilizar. Cada grupo de Auto Scaling se asocia a un proveedor de capacidad, y cada proveedor de capacidad tiene un solo grupo de Auto Scaling. Sin embargo, se pueden asociar muchos proveedores de capacidad a un clúster de ECS. Para escalar todo el clúster de forma automática, cada proveedor de capacidad administra el escalado de su grupo de Auto Scaling asociado.

Para obtener más información sobre el escalado automático de clústeres, consulte la publicación del blog de noticias de AWS [Escalado automático de clústeres de Amazon ECS](#).



De nuevo, considere la aplicación de foro monolítico que vimos anteriormente, en la que toda la aplicación se ejecuta como un único servicio. Para rediseñar esta aplicación utilizando una arquitectura de microservicios, puede ejecutar cada proceso de la aplicación como un servicio independiente en su propio contenedor. Con una arquitectura de microservicios, los servicios pueden escalar y actualizarse sin depender de los demás.

Para implementar la aplicación monolítica como una aplicación de microservicios, primero cree y etiquete una imagen para cada servicio. Luego, registre las imágenes con Amazon Elastic Container Registry (Amazon ECR).

Descomposición de monolitos. Paso 2: Crear la definición de tareas de servicio y grupos objetivo

Definición de tareas de servicio

- Tipo de inicio = [EC2 o Fargate]
- Nombre = [nombre-del-servicio]
- Imagen = [URL del repositorio de ECR del servicio]:versión
- CPU = [256]
- Memoria = [256]
- Puerto del contenedor = [3000]
- Puerto de host = [0]

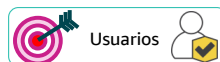
Grupo de destino del servicio

- Nombre = [nombre-del-servicio]
- Protocolo = [HTTP]
- Puerto = [80]
- VPC = [nombre-de-la-vpc]

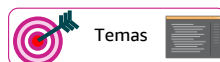


Amazon
ECS

Clúster



Usuarios



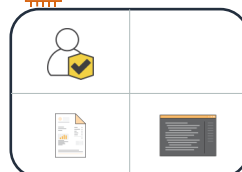
Temas



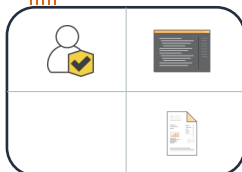
Mensajes

Grupos objetivo

Instancia de EC2 con
contenedores de servicio



Instancia de EC2 con
contenedores de servicio



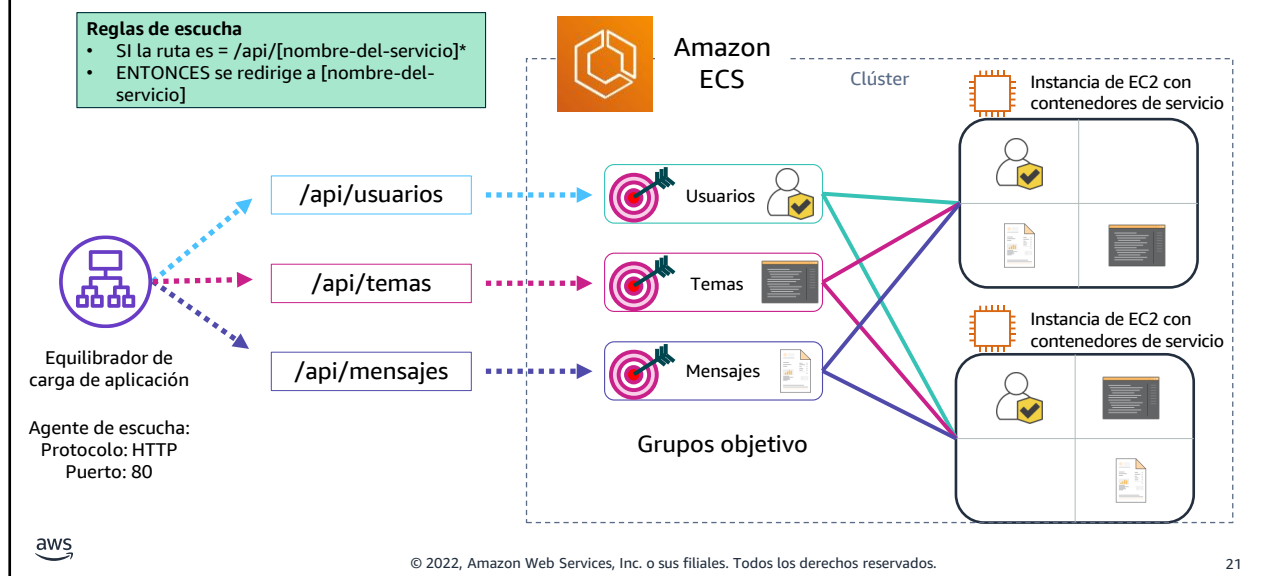
© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

20

A continuación, elija el tipo de lanzamiento y cree un nuevo servicio para cada pieza de la aplicación monolítica original. Amazon ECS implementa cada servicio en su propio contenedor en un clúster de ECS.

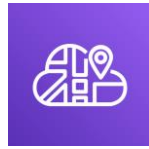
A continuación, cree un grupo objetivo para cada servicio. El grupo objetivo rastrea las instancias y puertos de cada contenedor que se está ejecutando para ese servicio.

Descomposición de monolitos. Paso 3: Conectar el equilibrador de carga a los servicios



Por último, cree un equilibrador de carga de aplicación y configure reglas de escucha para conectar los servicios. El agente de escucha comprueba las solicitudes de conexión entrantes al equilibrador de carga y utiliza las reglas para enrutar el tráfico de forma adecuada. En el ejemplo, el agente de escucha del equilibrador de carga de aplicación escucha las peticiones de servicio HTTP en el puerto 80 y las enruta al servicio correspondiente.

Herramientas para crear arquitecturas de microservicios de alta disponibilidad



AWS Cloud Map

- Es un servicio de descubrimiento completamente administrado para recursos en la nube
- Se puede utilizar para definir nombres personalizados de los recursos de la aplicación
- Mantiene actualizada la ubicación de los recursos que se modifican dinámicamente, lo que aumenta la disponibilidad de las aplicaciones.



AWS App Mesh

- Captura métricas, registros y rastreos de todos sus microservicios
- Permite exportar estos datos a Amazon CloudWatch, AWS X-Ray y herramientas compatibles con la red de socios de AWS (APN) y la comunidad
- Permite controlar los flujos de tráfico entre microservicios para garantizar que los servicios tengan alta disponibilidad



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

22

AWS Cloud Map y AWS App Mesh son dos herramientas que pueden ayudarle a crear arquitecturas de microservicios de alta disponibilidad.

[AWS Cloud Map](#) es un servicio de descubrimiento completamente administrado para recursos en la nube. Puede utilizarlo para definir nombres personalizados para los recursos de su aplicación (como bases de datos, colas, microservicios y otros recursos en la nube). AWS Cloud Map mantiene actualizada la ubicación de estos recursos que se modifican dinámicamente. Este mantenimiento de la ubicación aumenta la disponibilidad de su aplicación porque su servicio web siempre descubre las ubicaciones más actualizadas de sus recursos. Puede agregar y registrar cualquier recurso con una intervención manual mínima del mapeo. AWS Cloud Map le ayuda con el descubrimiento de servicios, la integración continua y la supervisión del estado de sus microservicios y aplicaciones.

Para obtener más información sobre AWS Cloud Map, lea esta publicación del blog [Código abierto de AWS](#). Para obtener más información sobre cómo puede utilizar AWS Cloud Map para permitir que sus servicios en contenedores se descubran y conecten entre sí, lea [AWS Fargate, Amazon EKS y Amazon ECS ahora se integran con AWS Cloud Map](#).

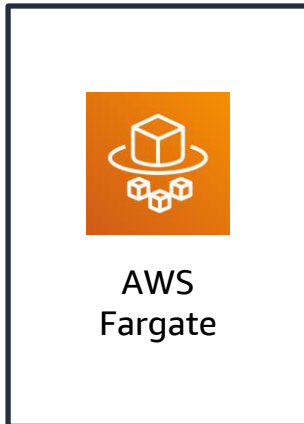
Cuando cree sus definiciones de tareas, puede habilitar la integración de App Mesh. [AWS App Mesh](#) captura métricas, registros y rastreos de todos sus microservicios. Puede exportar estos datos a Amazon CloudWatch, AWS X-Ray y herramientas compatibles con la red de socios de AWS (APN) y la comunidad para la supervisión y rastreo. AWS App Mesh también le permite controlar cómo fluye el tráfico entre los microservicios para garantizar que todos los servicios dispongan de alta disponibilidad durante las implementaciones, después de los errores y a

medida que se escala la aplicación.

App Mesh le permite configurar microservicios para que se conecten directamente entre sí a través de un proxy en lugar de necesitar un código dentro de la aplicación o utilizar un equilibrador de carga. App Mesh utiliza Envoy, un proxy de malla de servicios de código abierto, que se implementa junto a sus contenedores de microservicios.

Para obtener más información sobre AWS Cloud Map y AWS App Mesh, consulte este video de [AWS en YouTube](#).

AWS Fargate



- Es un **servicio de contenedores** completamente administrado
- Funciona con **Amazon Elastic Container Service (Amazon ECS)** y **Amazon Elastic Kubernetes Service (Amazon EKS)**
- Aprovisiona, administra y escala sus clústeres de contenedores
- Administra el entorno en tiempo de ejecución
- Brinda escalado automático



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

23

En esta sección, aprendió que Amazon ECS ofrece dos tipos de lanzamiento: EC2 y Fargate.

[AWS Fargate](#) es un servicio de contenedores completamente administrado que funciona tanto con Amazon ECS como con Amazon Elastic Kubernetes Service (Amazon EKS). Le permite ejecutar contenedores sin necesidad de administrar servidores o clústeres. Con AWS Fargate, ya no es necesario aprovisionar, configurar ni escalar clústeres de máquinas virtuales para ejecutar contenedores. Por lo tanto, no es necesario elegir tipos de servidores, decidir cuándo escalar los clústeres u optimizar conjuntos de clústeres. AWS Fargate reduce la necesidad de interactuar con servidores ni clústeres, ni siquiera pensar en ellos. Fargate le permite concentrarse en el diseño y la creación de aplicaciones, en lugar de administrar la infraestructura que las ejecuta.

Conclusiones importantes de la Sección 3



- **Amazon ECS** es un servicio de administración de contenedores altamente escalable y de alto rendimiento. Admite contenedores de Docker y permite ejecutar aplicaciones fácilmente en un clúster administrado de instancias de Amazon EC2.
- **El escalado automático** de clústeres le ofrece más control sobre cómo escalar tareas en un clúster.
- **AWS Cloud Map** permite definir nombres personalizados para los recursos de la aplicación. Mantiene actualizada la ubicación de estos recursos que se modifican dinámicamente.
- **AWS App Mesh** es una malla de servicios que proporciona redes a nivel de aplicación. Permite que sus servicios se comuniquen fácilmente entre sí a través de diversos tipos de infraestructuras de cómputo.
- **AWS Fargate** es un servicio de contenedores completamente administrado que permite ejecutar contenedores sin necesidad de administrar servidores o clústeres.

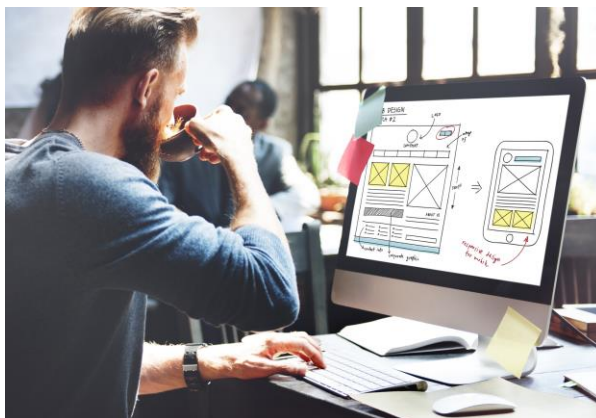
© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

24

Entre los puntos clave de esta sección del módulo, se incluyen los siguientes:

- Amazon ECS es un servicio de administración de contenedores altamente escalable y de alto rendimiento. Admite contenedores de Docker y permite ejecutar aplicaciones fácilmente en un clúster administrado de instancias de Amazon EC2.
- El escalado automático de clústeres le ofrece más control sobre cómo escalar tareas con un clúster.
- AWS Cloud Map permite definir nombres personalizados para los recursos de la aplicación. Mantiene actualizada la ubicación de estos recursos que se modifican dinámicamente.
- AWS App Mesh es una malla de servicios que proporciona redes a nivel de aplicación para facilitar la comunicación entre los servicios a través de varios tipos de infraestructura de cómputo.
- AWS App Mesh es una malla de servicios que proporciona redes a nivel de aplicación. Permite que sus servicios se comuniquen fácilmente entre sí a través de diversos tipos de infraestructuras de cómputo.
- AWS Fargate es un servicio de contenedores completamente administrado que permite ejecutar contenedores sin necesidad de administrar servidores o clústeres.

Módulo 13. Laboratorio guiado 1: División de una aplicación monolítica Node.js en microservicios (laboratorio opcional)



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

25

Podría completar el módulo 13. Laboratorio guiado 1: División de una aplicación monolítica Node.js en microservicios. Este laboratorio es opcional.

Laboratorio guiado 1: Tareas

1. Preparar el entorno de desarrollo de AWS Cloud9
2. Ejecutar la aplicación monolítica en un servidor Node.js básico
3. Colocar en contenedores el monolito para Amazon ECS
4. Implementar el monolito en Amazon ECS
5. Refactorizar el monolito en microservicios dentro de contenedores



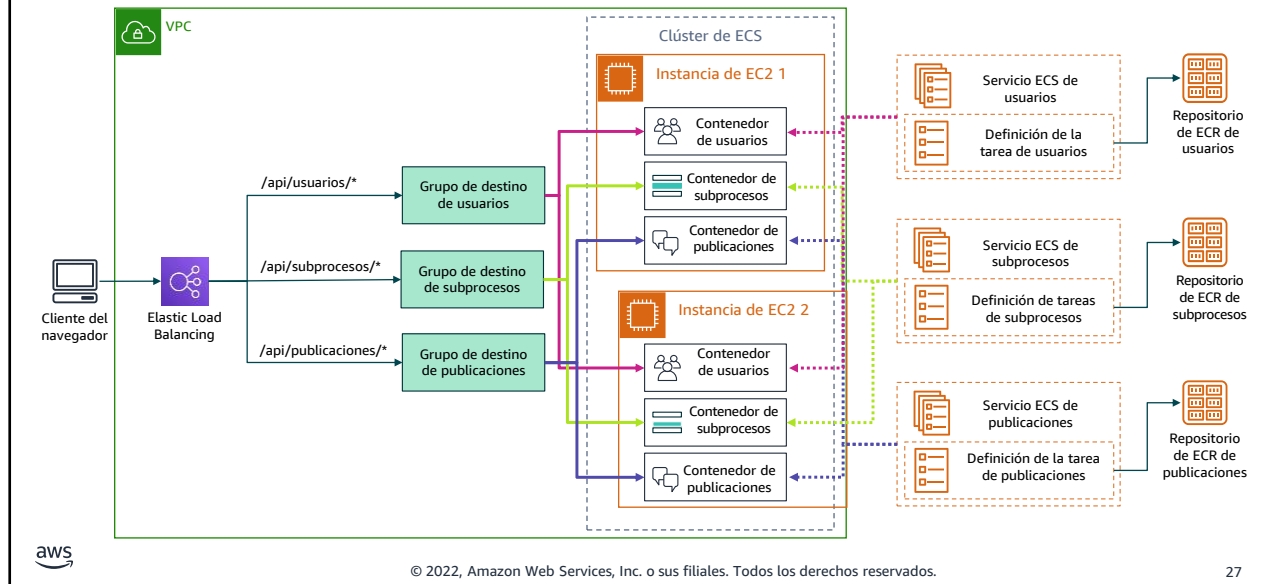
© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

26

En este laboratorio guiado, realizará las siguientes tareas:

1. Preparar el entorno de desarrollo de AWS Cloud9
2. Ejecutar la aplicación monolítica en un servidor Node.js básico
3. Colocar en contenedores el monolito para Amazon ECS
4. Implementar el monolito en Amazon ECS
5. Refactorizar el monolito en microservicios dentro de contenedores

Laboratorio guiado 1: Producto final



El diagrama resume lo que creará después de completar el laboratorio.



~ 3 horas



Inicio del módulo 13. Laboratorio guiado 1: División de una aplica- ción monolítica Node.js en microservicios

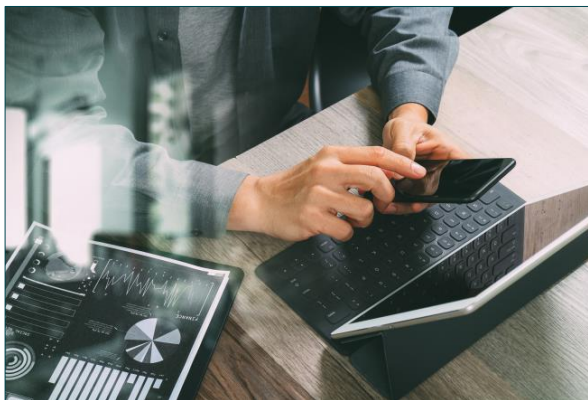


© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

28

Ahora es momento de iniciar el laboratorio guiado opcional.

Análisis posterior del laboratorio guiado 1: Aprendi- zajes clave



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

29

El instructor puede optar por dirigir una conversación sobre los puntos clave de este laboratorio guiado después de que lo haya completado.

Sección 4: Presentación de las arquitecturas sin servidor

Módulo 13: Creación de microservicios y arquitecturas sin servidor



Introducción a la sección 4: Presentación de las arquitecturas sin servidor.

¿Qué significa sin servidor?

Una forma de crear y ejecutar aplicaciones y servicios sin tener que preocuparse por los servidores



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

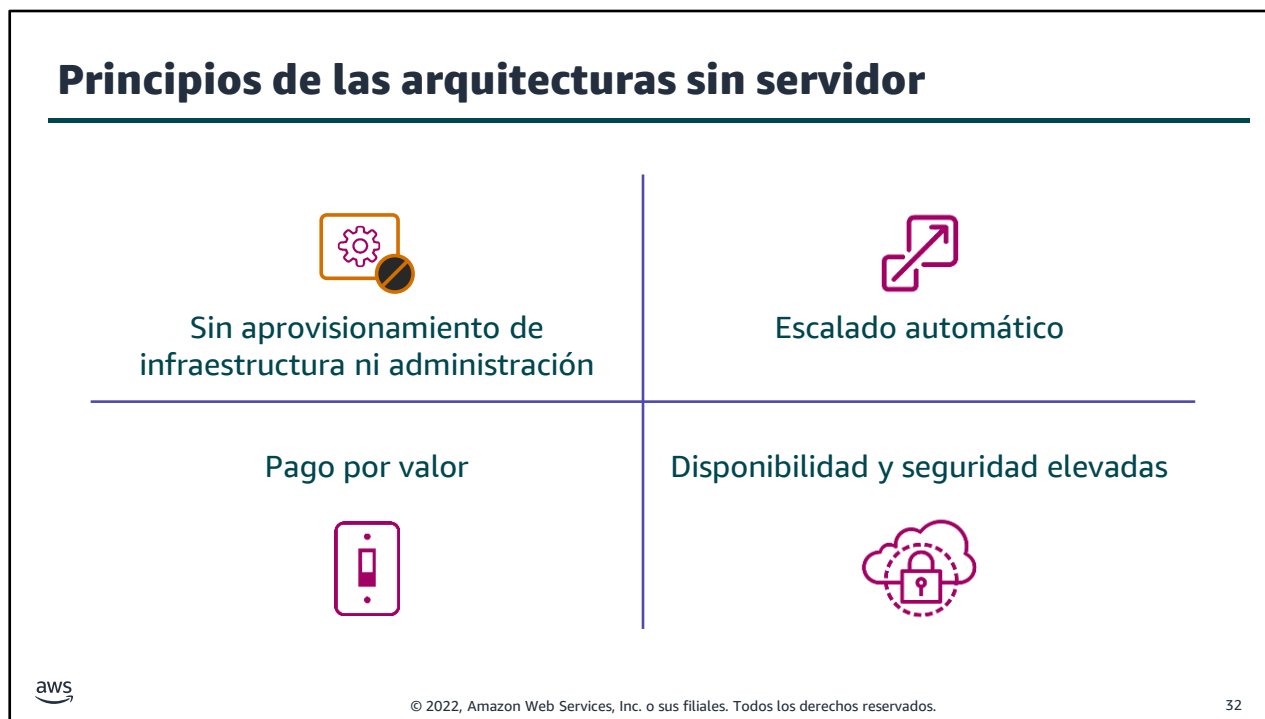
31

Hasta ahora, ha aprendido que puede utilizar Amazon ECS para crear sus aplicaciones de microservicios utilizando contenedores. Amazon ECS es un servicio de coordinación de contenedores en el que se administra el código de la aplicación, las integraciones de origen de datos, la configuración de seguridad, las actualizaciones, la configuración de red, el firewall y las tareas de administración. También ha aprendido que puede utilizar el tipo de lanzamiento de Fargate para hospedar su clúster en una infraestructura *sin servidor* que administra Amazon ECS.

Pero, ¿qué significa sin servidor?

Un modelo *sin servidor* es la arquitectura nativa de la nube que le permite trasladar más responsabilidades operativas a AWS, lo que puede aumentar la agilidad e innovación. Gracias al cómputo sin servidor, puede crear y ejecutar aplicaciones y servicios sin tener que preocuparse por los servidores. La aplicación se sigue ejecutando en servidores. Sin embargo, AWS realiza todas las tareas de administración de los servidores, como el aprovisionamiento de servidores o clústeres, la aplicación de parches, el mantenimiento del sistema operativo y el aprovisionamiento de capacidad.

Principios de las arquitecturas sin servidor



Los principios que definen el modelo operativo sin servidor son los siguientes:

- Sin infraestructura que aprovisionar o administrar (no hay servidores para aprovisionar, operar o aplicar parches)
- Se escala automáticamente por unidad de consumo (se escala por unidad de trabajo o consumo en lugar de por unidad de servidor)

Modelo de precios de *pago por valor* (solo pagará por la duración de la ejecución de un recurso, en lugar de hacerlo por unidad de servidor)

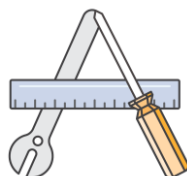
- Tiene disponibilidad prediseñada y tolerancia a errores (no es necesario diseñar la disponibilidad, ya que está integrada en el servicio)

Para obtener más información sobre el modelo sin servidor, consulte [este sitio web de AWS](#).

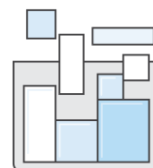
Beneficios del modelo sin servidor



Menor costo total
de propiedad



Céntrese en su
aplicación, no en
la configuración



Cree aplicaciones
de **microservicios**

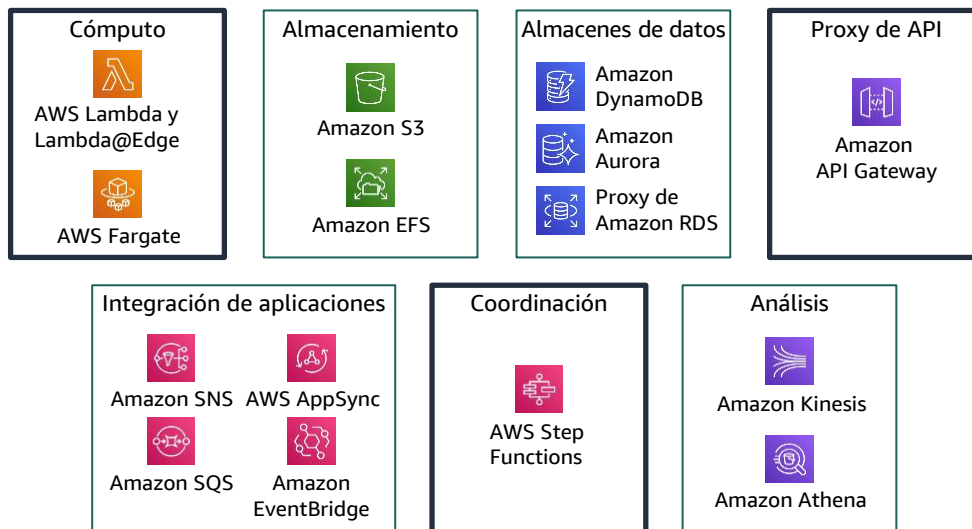


© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

33

El modelo sin servidor le permite crear [aplicaciones modernas](#) con mayor agilidad y menor costo total de propiedad (TCO). Al utilizar una arquitectura sin servidor, puede centrarse en su producto principal. No tiene que preocuparse de administrar ni operar servidores ni tiempos de ejecución en la nube o en las instalaciones. Esta reducción de la sobrecarga le permite recuperar tiempo y energía, que puede dedicar al desarrollo de productos escalables y fiables. Por último, las arquitecturas sin servidor permiten crear aplicaciones de microservicios.

Ofertas sin servidor de AWS



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

34

AWS tiene muchas ofertas que puede utilizar para crear arquitecturas sin servidor en AWS. Hasta ahora en este curso, ha aprendido sobre varias de ellas.

El resto de este módulo se centra en cómo puede utilizar AWS Lambda, Amazon API Gateway y AWS Step Functions para crear arquitecturas sin servidor.

Conclusiones importantes de la Sección 4



- El **cómputo sin servidor** le permite crear y ejecutar aplicaciones y servicios sin aprovisionar ni administrar servidores
- Las arquitecturas sin servidor ofrecen los siguientes **beneficios**:
 - Menor costo total de propiedad (TCO)
 - Puede centrarse en su aplicación
 - Puede utilizarlos para crear aplicaciones de microservicios

© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

35

Entre los puntos clave de esta sección del módulo, se incluyen los siguientes:

- El **cómputo sin servidor** le permite crear y ejecutar aplicaciones y servicios sin aprovisionar ni administrar servidores
- Las arquitecturas sin servidor ofrecen los siguientes beneficios:
 - Menor TCO
 - Puede centrarse en su aplicación
 - Puede utilizarlos para crear aplicaciones de microservicios

Sección 5: Creación de arquitecturas sin servidor con AWS Lambda

Módulo 13: Creación de microservicios y arquitecturas sin servidor



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

Introducción a la sección 5: Creación de arquitecturas sin servidor con AWS Lambda.

AWS Lambda



- Es un servicio de cómputo **completamente administrado**
- Ejecuta su código según una programación o como **respuesta a eventos** (por ejemplo, cambios en un bucket de Amazon S3 o en una tabla de Amazon DynamoDB)
- Compatible con Java, Go, PowerShell, Node.js, C#, Python, Ruby y API de tiempo de ejecución
- Puede funcionar en ubicaciones perimetrales más cercanas a sus usuarios



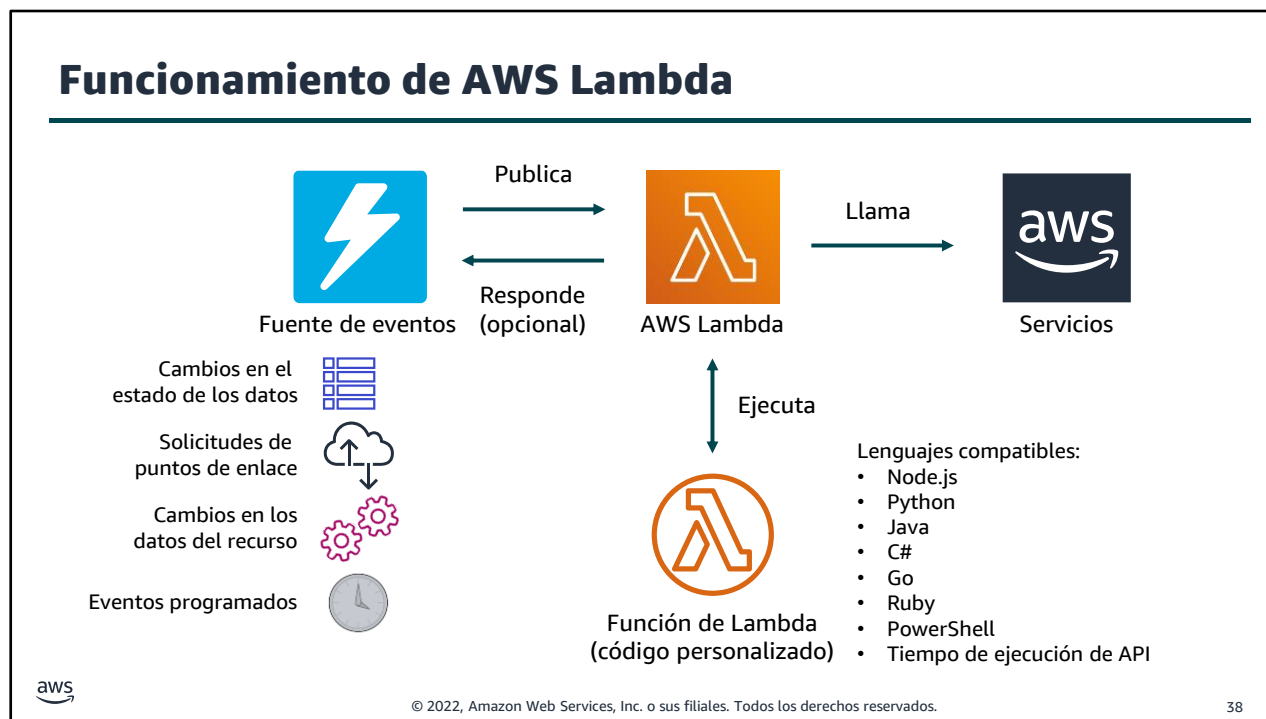
© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

37

[AWS Lambda](#) es un servicio de cómputo completamente administrado que ejecuta código en respuesta a eventos y administra automáticamente los recursos de cómputo subyacentes. Lambda ejecuta el código en una infraestructura de cómputo de alta disponibilidad y realiza toda la administración de los recursos de cómputo, incluido el mantenimiento de servidores y sistemas operativos, el aprovisionamiento de la capacidad, el escalado automático, y la supervisión y el registro de códigos.

AWS Lambda admite de forma nativa código Java, Go, PowerShell, Node.js, C#, Python y Ruby, y proporciona una API de tiempo de ejecución que le permite utilizar otros lenguajes de programación para crear sus funciones.

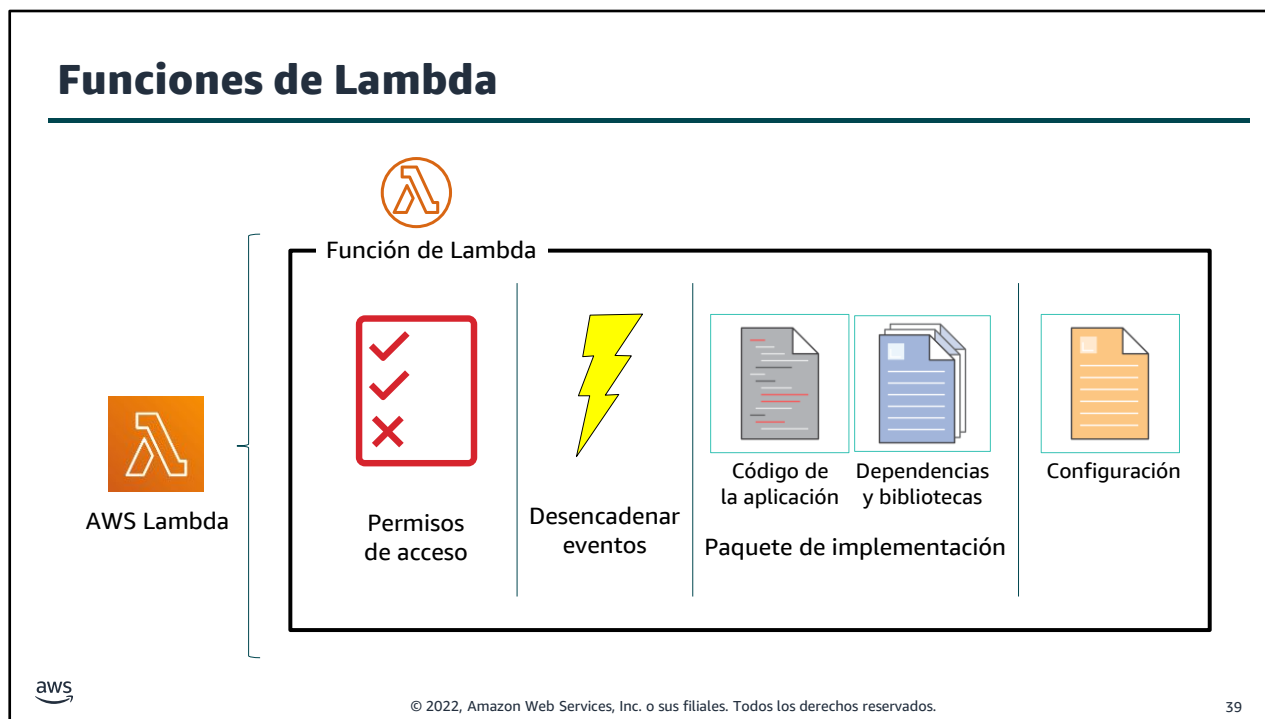
[Lambda@Edge](#) es una función de Amazon CloudFront que le permite ejecutar el código más cerca de los usuarios de su aplicación, lo que mejora el rendimiento y reduce la latencia. Lambda@Edge ejecuta su código como respuesta a eventos generados por la red de entrega de contenido (content delivery network, CDN) de Amazon CloudFront. Lambda@Edge le permite ejecutar funciones de Lambda de Node.js y Python para personalizar el contenido que entrega Amazon CloudFront. Para obtener información sobre cómo agregar encabezados de respuesta de seguridad HTTP, lea esta [publicación del blog Redes y entrega de contenido de AWS](#).



AWS Lambda se integra con otros servicios de AWS para invocar funciones de Lambda. Una función de *Lambda* es código personalizado que se escribe en uno de los lenguajes compatibles con Lambda. Puede configurar desencadenadores para invocar una función como respuesta a eventos del ciclo de vida de los recursos, responder solicitudes HTTP entrantes, consumir eventos de una cola o ejecutarse según una programación.

Una *fuentes de eventos* es la entidad que publica el evento en Lambda. La función de Lambda procesa el evento y Lambda ejecuta la función de Lambda por usted.

Las funciones de Lambda son *sin estado*, es decir, no tienen afinidad con la infraestructura subyacente. Lambda puede iniciar rápidamente tantas copias de la función como sea necesario para escalar a la velocidad de los eventos entrantes.



Cuando se crea una función de Lambda, se definen los permisos para la función y se especifica qué eventos la desencadenan. También se crea un paquete de implementación que incluye el código de la aplicación y las dependencias y bibliotecas necesarias para ejecutar el código. Por último, configure los parámetros de tiempo de ejecución, como la memoria, el tiempo de espera y la simultaneidad. Cuando se invoca su función, Lambda ejecutará un entorno basado en el tiempo de ejecución y las opciones de configuración que haya seleccionado.

Para obtener más información sobre cómo se ejecuta AWS Lambda, consulte la [documentación de AWS](#).

Anatomía de una función de Lambda

Handler()

Función que se ejecutará después de la invocación

Objeto de evento

Datos enviados durante la invocación de la función de Lambda

Objeto del contexto

Métodos disponibles para interactuar con la información de tiempo de ejecución (ID de solicitud, grupo de registro, más)

```
import json

def lambda_handler(event, context):
    # TODO implement
    return {
        'statusCode': 200,
        'body': json.dumps('Hello world')
    }
```



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

40

Cuando se invoca una función de Lambda, el código comienza a ejecutarse en el controlador. El controlador es un método o función de código específico que crea e incluye en su paquete. Se especifica el controlador cuando se crea una función de Lambda. Cada lenguaje compatible tiene sus propios requisitos sobre cómo se puede definir un controlador de funciones y hacer referencia a este dentro del paquete. Después de que se invoque correctamente el controlador en su función de Lambda, el entorno en tiempo de ejecución pertenece al código que escribió.

El controlador siempre toma dos objetos: el objeto de evento y el objeto de contexto.

El objeto de evento proporciona información acerca del evento que desencadenó la función de Lambda. Este evento podría ser un objeto predefinido generado por un servicio de AWS o un objeto personalizado definido por el usuario como una cadena serializable. Un ejemplo de este tipo de cadena podría ser un POJO (Plain Old Java Object) o una secuencia JSON.

El contenido del objeto de evento incluye todos los datos y metadatos que la función de Lambda necesita para impulsar la lógica. El contenido y la estructura del objeto de evento difieren según la fuente del evento que lo haya creado. Por ejemplo, un evento que crea API Gateway tendrá detalles relacionados con la solicitud HTTPS que realizó el cliente, como la ruta, la cadena de consulta y el cuerpo de la solicitud. Sin embargo, un evento creado por Amazon incluye detalles sobre el bucket y el nuevo objeto.

AWS genera el objeto de contexto y proporciona metadatos sobre el entorno en tiempo de ejecución. El objeto de contexto permite que el código de función interactúe con el entorno en tiempo de ejecución de Lambda. Los contenidos y la estructura del objeto de contexto varían en función del tiempo de ejecución del lenguaje que use la función de Lambda.

Sin embargo, como mínimo, el objeto de contexto contiene:

- `awsRequestId`: esta propiedad se utiliza para realizar un seguimiento de las invocaciones específicas de una función de Lambda (importante para informar los errores o al contactar con AWS Support)
- `logStreamName`: indica el flujo de registro de CloudWatch al que se enviarán los enunciados de registro
- `getRemainingTimeInMillis()`: este método devuelve el número de milisegundos que faltan antes de que se agote el tiempo de ejecución de su función

Configuración y facturación de las funciones de Lambda

Memoria: El costo por 1 ms de duración de la función aumenta a medida que lo hace la memoria.

Tiempo de espera: Puede controlar la duración máxima de su función.

Precios: Se le cobrará en función del número de solicitudes y la duración.



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

41

La memoria y el tiempo de espera son configuraciones que determinan el rendimiento de la función de Lambda. Estas configuraciones afectan a su facturación. Con AWS Lambda, se le cobrará en función del número de solicitudes de las funciones (el número total de solicitudes en todas sus funciones) y la duración (el tiempo que tarde en ejecutarse el código). El precio depende de la cantidad de memoria que asigne a su función.

Memoria: especifica la cantidad de memoria que desea asignar a su función de Lambda. Lambda asigna capacidad de procesamiento de CPU proporcional a la memoria. Lambda tiene un precio que hace que el costo por 1 ms de duración de la función aumente a medida que aumenta la configuración de la memoria. Por ejemplo, supongamos que tiene una función de Lambda con 256 MB de memoria y que se ejecuta durante 110 milisegundos. Esta función costará el doble que una función de Lambda con 128 MB de memoria que se ejecute durante el mismo tiempo.

Tiempo de espera: puede controlar la duración máxima de su función configurando el tiempo de espera. Puede establecer el valor de tiempo de espera de una función en cualquier valor hasta un máximo de 15 minutos. Cuando se alcanza el tiempo de espera especificado, AWS Lambda detiene la ejecución de su función de Lambda. Utilizar un tiempo de espera puede evitar mayores costos derivados de funciones de ejecución prolongada. Debe encontrar el equilibrio adecuado entre no prolongar demasiado la función y poder terminarla en circunstancias normales.

Siga estas prácticas recomendadas:

- Pruebe el rendimiento de la función de Lambda para garantizar la elección de la configuración de capacidad de memoria óptima. Puede ver el uso de memoria de su función en los registros de Amazon CloudWatch.

- Pruebe su función de Lambda para analizar el tiempo de ejecución y determinar el mejor valor de tiempo de espera. Esto es especialmente importante cuando la función de Lambda realiza llamadas de red a los recursos que podrían no ser capaces de gestionar el escalado de las funciones de Lambda.

Consulte los siguientes recursos para obtener información sobre:

- [Límites de AWS Lambda](#)
- [Precios de AWS Lambda](#)

Demostración: Creación de una función de AWS Lambda

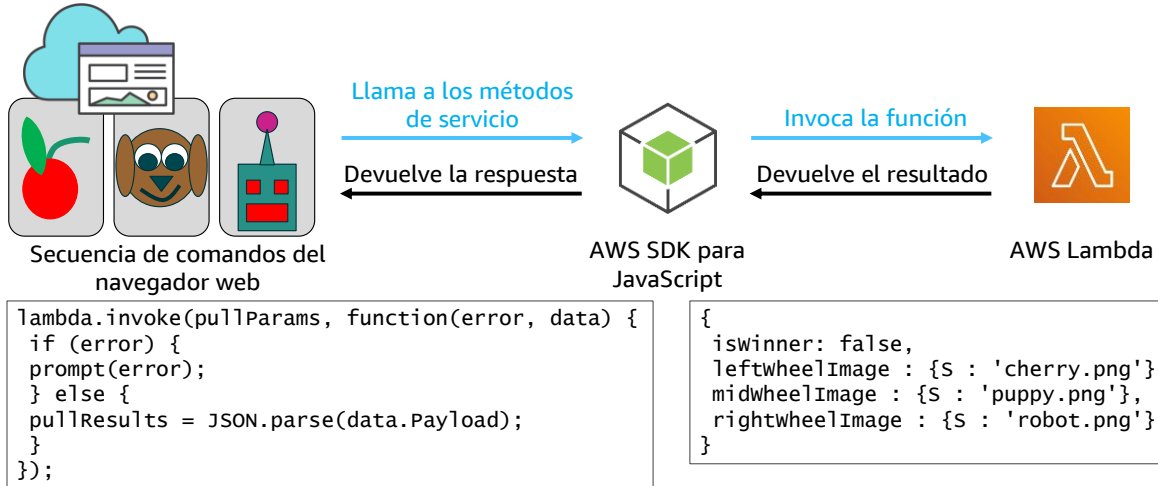


© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

42

Ahora, el instructor puede hacer una demostración de cómo crear una función de AWS Lambda.

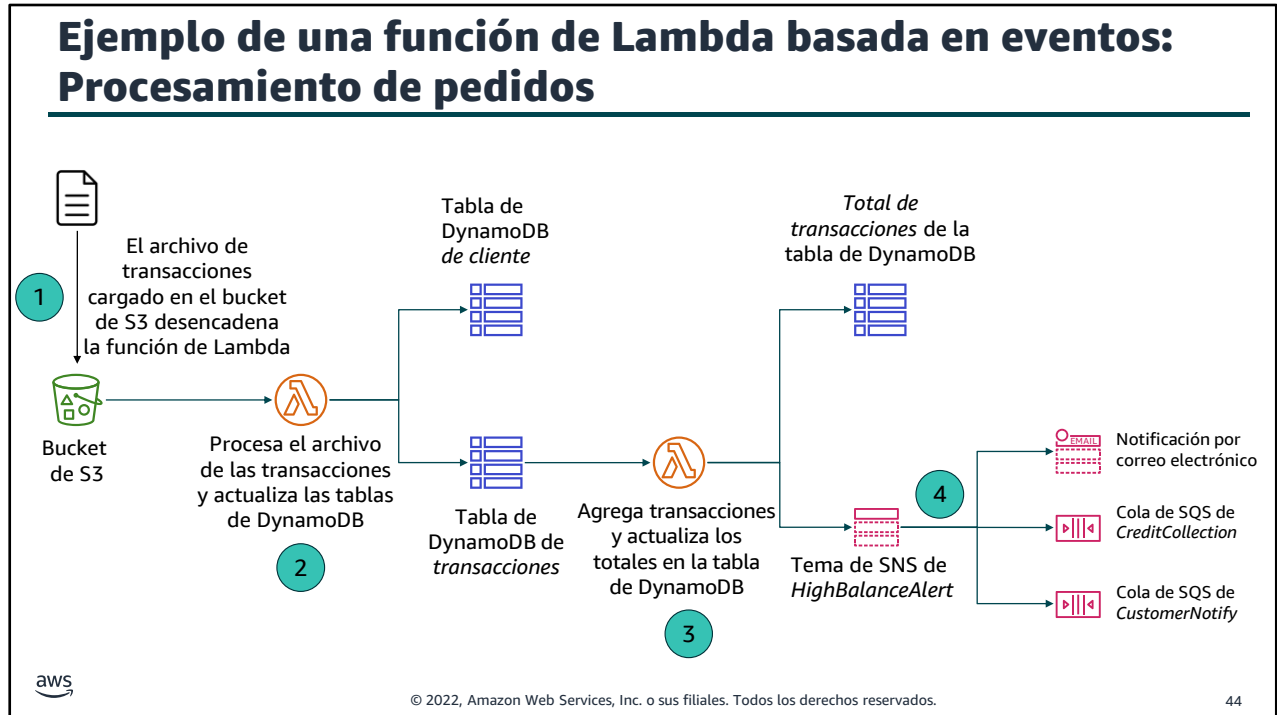
Ejemplo de AWS Lambda: Juego de navegador de máquina tragamonedas simulada



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

43

Puede crear funciones de Lambda para realizar diversas tareas. En este ejemplo se muestra un juego basado en navegador que simula una máquina tragamonedas. El juego invoca una función de Lambda que genera los resultados aleatorios de cada tiro en la máquina tragamonedas. La función devuelve esos resultados como los nombres de archivo de las imágenes utilizadas para mostrar el resultado. Las imágenes se almacenan en un bucket de Amazon S3 que está configurado para funcionar como un host web estático de los activos HTML, CSS y otros que se necesitan para presentar la experiencia de la aplicación.



En este ejemplo se muestra cómo se puede utilizar Lambda en una solución para el procesamiento de pedidos.

En esta arquitectura:

1. Un cliente carga un archivo de transacciones a un bucket de S3, lo que desencadena la ejecución de una función de Lambda.
2. Una función de Lambda procesa el archivo de transacciones y actualiza las tablas *Customer* y *Transactions* de DynamoDB.
3. Los cambios en la tabla *Transactions* de DynamoDB activan una segunda función de Lambda para agregar las transacciones y actualizar los totales en la tabla *Transaction total* de DynamoDB. También envía un mensaje *HighBalancerAlert* al tema de SNS.
4. El tema de SNS *HighBalancerAlert* envía una notificación por correo electrónico al cliente y actualiza las colas de SQS *CreditCollection* y *CustomerNotify* para el procesamiento del pago.

Capas de Lambda



- Habilitan funciones para compartir código fácilmente: puede cargar una capa una vez y utilizarla como referencia en cualquier función
- Promueven la división de responsabilidades: los desarrolladores pueden iterar más rápido en la escritura de la lógica empresarial
- Permiten reducir los paquetes de implementación
- Límites:
 - Hasta cinco capas
 - 250 MB



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

45

Cuando se crean aplicaciones sin servidor, es común tener código que se comparte a través de las funciones de Lambda. Puede ser código personalizado que utilizan dos o más funciones, o una biblioteca estándar que se agrega para simplificar la implementación de su lógica empresarial.

Antes, se empaquetaba e implementaba este código compartido junto con todas las funciones que lo utilizaban. Ahora, puede configurar su función de Lambda para incorporar código y contenido adicionales como capas. Una capa es un archivo .zip que contiene bibliotecas, un tiempo de ejecución personalizado u otras dependencias.

Con las capas de Lambda, las funciones pueden compartir código. Los desarrolladores utilizan capas para cargar código una vez y reutilizarlo varias veces. Con las capas, puede utilizar las bibliotecas en la función sin necesidad de incluirlas en el paquete de implementación.

Compartir el código de esta forma puede ayudar a promover la división de responsabilidades. Una persona puede encargarse de gestionar la biblioteca principal. Otra persona puede ocuparse de utilizar y crear sobre el código de la biblioteca para desarrollar la lógica de la aplicación.

Las capas permiten reducir el tamaño del paquete de implementación, lo que facilita el desarrollo.

Una función puede utilizar hasta cinco capas a la vez. El tamaño total descomprimido de la función y todas las capas no puede superar el límite del tamaño del paquete de implementación sin comprimir, 250 MB.

Para obtener más información sobre las capas, consulte [Capas de AWS Lambda](#).

Demostración: Uso de AWS Lambda con Amazon S3



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

46

Ahora, el instructor puede hacer una demostración de cómo configurar un evento de Amazon S3 para desencadenar una función de Lambda.

Conclusiones importantes de la Sección 5



- **Lambda** es un servicio de cómputo sin servidor que proporciona tolerancia a errores integrada y escalado automático
- Una **función de Lambda** es un código personalizado que se escribe para procesar eventos
- Un **controlador** invoca una función de Lambda, que toma como parámetros un **objeto de evento** y un **objeto de contexto**
- Un **origen de eventos** es un servicio de AWS o una aplicación creada por el desarrollador que desencadena una función de Lambda para que se ejecute
- **Las capas** de Lambda permiten que las funciones compartan código y limitan el tamaño de los paquetes de implementación

© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

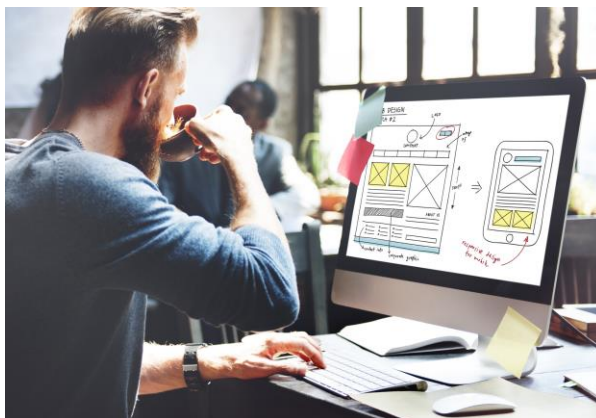
51

Entre los puntos clave de esta sección del módulo, se incluyen los siguientes:

- Lambda es un servicio de cómputo sin servidor que proporciona tolerancia a errores integrada y escalado automático.
- Una función de Lambda es un código personalizado que se escribe para procesar eventos.
- Un controlador invoca una función de Lambda, que toma como parámetros un objeto de evento y un objeto de contexto.
- Un origen de eventos es un servicio de AWS o una aplicación creada por el desarrollador que desencadena una función de Lambda para que se ejecute.

Las capas de Lambda permiten que las funciones compartan código y limitan el tamaño de los paquetes de implementación.

Módulo 13: Laboratorio guiado 2: Implementación de una arquitectura sin servidor en AWS



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

52

Ahora completará el módulo 13. Laboratorio guiado 2: Implementación de una arquitectura sin servidor en AWS.

Laboratorio guiado 2: Tareas

1. Crear una función de Lambda para la carga de datos
2. Configurar un evento de Amazon S3
3. Probar el proceso de carga
4. Configurar las notificaciones
5. Crear una función de Lambda para enviar notificaciones
6. Probar el sistema



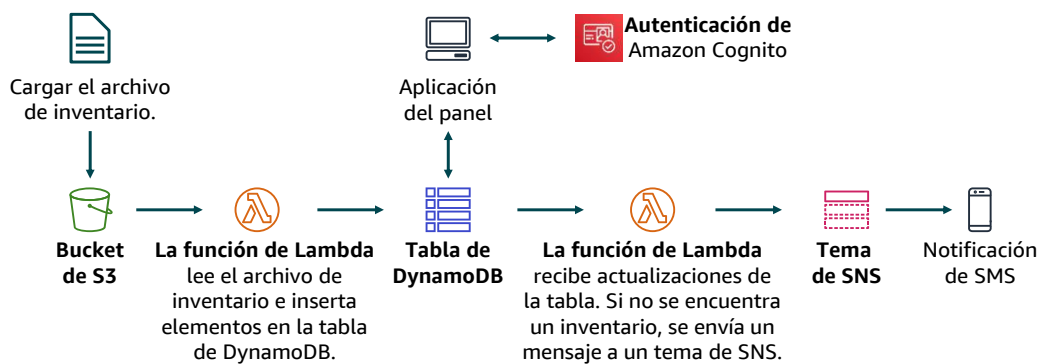
© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

53

En este laboratorio guiado, realizará las siguientes tareas:

1. Crear una función de Lambda para la carga de datos
2. Configurar un evento de Amazon S3
3. Probar el proceso de carga
4. Configurar las notificaciones
5. Crear una función de Lambda para enviar notificaciones
6. Probar el sistema

Laboratorio guiado 2: Producto final



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

54

El diagrama resume lo que creará después de completar el laboratorio.



~ 40 minutos



Inicio del módulo 13: Laboratorio guiado 2: Implementación de una arquitectura sin servidor en AWS



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

55

Es momento de iniciar el laboratorio guiado.

Análisis posterior del laboratorio guiado 2: Aprendizajes clave



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

56

El instructor puede optar por dirigir una conversación sobre los puntos clave de este laboratorio guiado después de que lo haya completado.

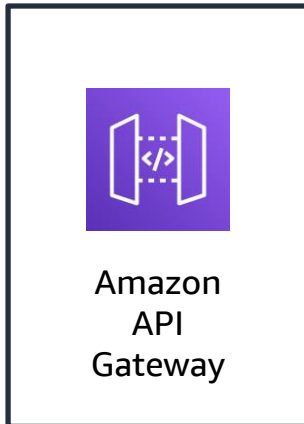
Sección 6: Ampliación de las arquitecturas sin servidor con Amazon API Gateway

Módulo 13: Creación de microservicios y arquitecturas sin servidor



Introducción a la sección 6: Ampliación de las arquitecturas sin servidor con Amazon API Gateway.

Amazon API Gateway



- Permite crear, publicar, mantener, supervisar y proteger API que funcionan como puntos de entrada a recursos backend para sus aplicaciones
- Gestiona hasta cientos de miles de llamadas API simultáneas
- Puede gestionar cargas de trabajo que se ejecutan en:
 - Amazon EC2
 - Lambda
 - Cualquier aplicación web
 - Aplicaciones de comunicación en tiempo real
- Puede alojar y utilizar varias versiones y fases de API



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

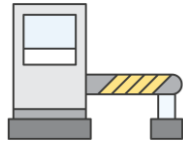
58

Amazon API Gateway es un servicio completamente administrado que le permite crear, publicar, mantener, supervisar y proteger API a cualquier escala. Puede utilizarlo para crear API de transferencia de estado representacional (RESTful) y WebSocket que funcionen como un punto de entrada para que las aplicaciones puedan acceder a los recursos de *backend*. De este modo, las aplicaciones pueden acceder a los datos, la lógica empresarial o la funcionalidad de sus servicios de *backend*. Estos servicios incluyen aplicaciones que se ejecutan en Amazon EC2, código que se ejecuta en Lambda, cualquier aplicación web o aplicaciones de comunicación en tiempo real.

API Gateway gestiona todas las tareas relacionadas con la aceptación y el procesamiento de hasta cientos de miles de llamadas API simultáneas. Estas llamadas pueden incluir la administración del tráfico, el control de la autorización y el acceso, la supervisión y la administración de versiones de la API. API Gateway no requiere pagos mínimos ni tiene costos iniciales. Solo deberá pagar por las llamadas a la API que reciba y la cantidad de datos transferidos. Con el modelo de precios por niveles de API Gateway, puede reducir sus costos a medida que aumenta el uso de la API.

Puede utilizar API Gateway para alojar varias versiones y fases de sus API.

Seguridad de Amazon API Gateway



Necesita
autorización



Aplica políticas
de recursos



Configuración
de limitación
controlada



Protección contra
ataques de denegación
de servicio distribuido
(DDoS) e inyección



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

59

Cuando publica sus API, se expone a posibles ataques que intenten explotar sus servicios. Con Amazon API Gateway, puede proteger sus API de varias formas.

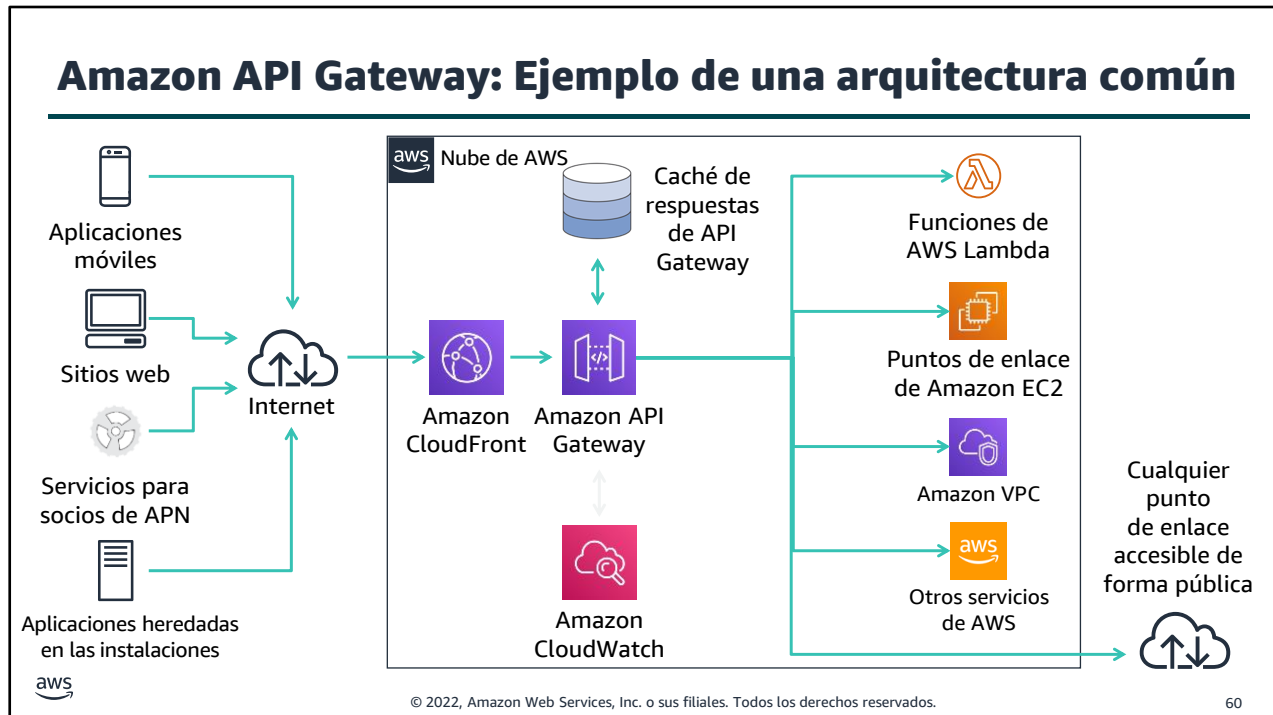
Con Amazon API Gateway, puede configurar sus métodos API de manera opcional para que necesiten autorización. Cuando configure un método de autorización, puede utilizar los autorizadores Lambda o AWS Signature Version 4 para que sean compatibles con su propia estrategia de autenticación de token portador. AWS Signature Version 4 es el proceso para agregar información de autenticación a las solicitudes de AWS que se envían a través de HTTP. Por seguridad, la mayoría de las solicitudes a AWS se deben firmar con una clave de acceso, que se compone de un ID de clave de acceso y una clave de acceso secreta. Utilice estas credenciales de AWS para firmar solicitudes a su servicio y autorizar el acceso, al igual que otros servicios de AWS. Puede recuperar credenciales temporales asociadas a un rol en su cuenta de AWS mediante Amazon Cognito. Un autorizador Lambda es una función de Lambda que autoriza el acceso a las API mediante una estrategia de autenticación de token portador como OAuth.

También puede aplicar una política de recursos a una API para restringir el acceso a una VPC de Amazon o a un punto de enlace de VPC específicos. Puede conceder acceso a una VPC de Amazon o un punto de enlace de VPC de una cuenta diferente a la API privada mediante una política de recursos.

Amazon API Gateway admite la configuración de limitación controlada para cada método o ruta de sus API. Puede establecer un límite de velocidad estándar y otro de ráfaga por segundo para cada método en sus API REST y cada ruta de las API WebSocket.

Además, puede utilizar AWS WAF para proteger sus API de API Gateway. [AWS WAF](#) es un firewall de aplicaciones web que protege sus aplicaciones web de las vulnerabilidades de seguridad más habituales que pueden afectar la disponibilidad, poner en riesgo la seguridad o consumir demasiados recursos.

Para obtener más información sobre cómo proteger sus API con Amazon API Gateway, consulte la [sección Seguridad y autorización](#) de las preguntas frecuentes de Amazon API Gateway.



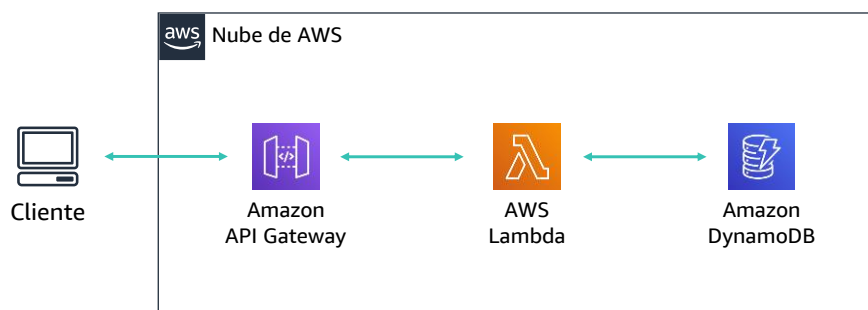
Puede utilizar Amazon API Gateway para suministrar la capa API a sus aplicaciones. A continuación, se muestra un ejemplo de una arquitectura común con Amazon API Gateway.

En este ejemplo, las aplicaciones de clientes *frontend* y los servicios de aplicación envían tráfico a API Gateway a través de Internet. Con frecuencia, Amazon CloudFront se utiliza para almacenar contenido estático en caché. API Gateway resume y expone las API que pueden llamar a varias aplicaciones de *backend*. Estas aplicaciones incluyen funciones de Lambda, contenedores de Docker que se ejecutan en instancias de EC2, redes virtuales privadas (VPC) o cualquier punto de enlace accesible de forma pública. Si es necesario, API Gateway puede almacenar la respuesta en caché. Por último, se pueden supervisar todas las llamadas API con Amazon CloudWatch.

Puede utilizar API Gateway con otros servicios administrados de AWS para crear *backends* sin servidor para sus aplicaciones. Por ejemplo, API Gateway puede enviar solicitudes a funciones de Lambda que ejecutan su código y generan respuestas. Incluso puede crear API que envíen solicitudes de proxy a otros servicios de AWS, como Amazon Simple Storage Service (Amazon S3), sin tener que escribir código. Puede poner en funcionamiento *backends* a escala de producción más rápidamente porque tiene menos código que escribir, y además delegó la carga operativa a los servicios de AWS.

Para ver un ejemplo de cómo utilizar API Gateway con AWS Lambda, consulte esta [publicación del blog de AWS](#).

Ejemplo: microservicios RESTful



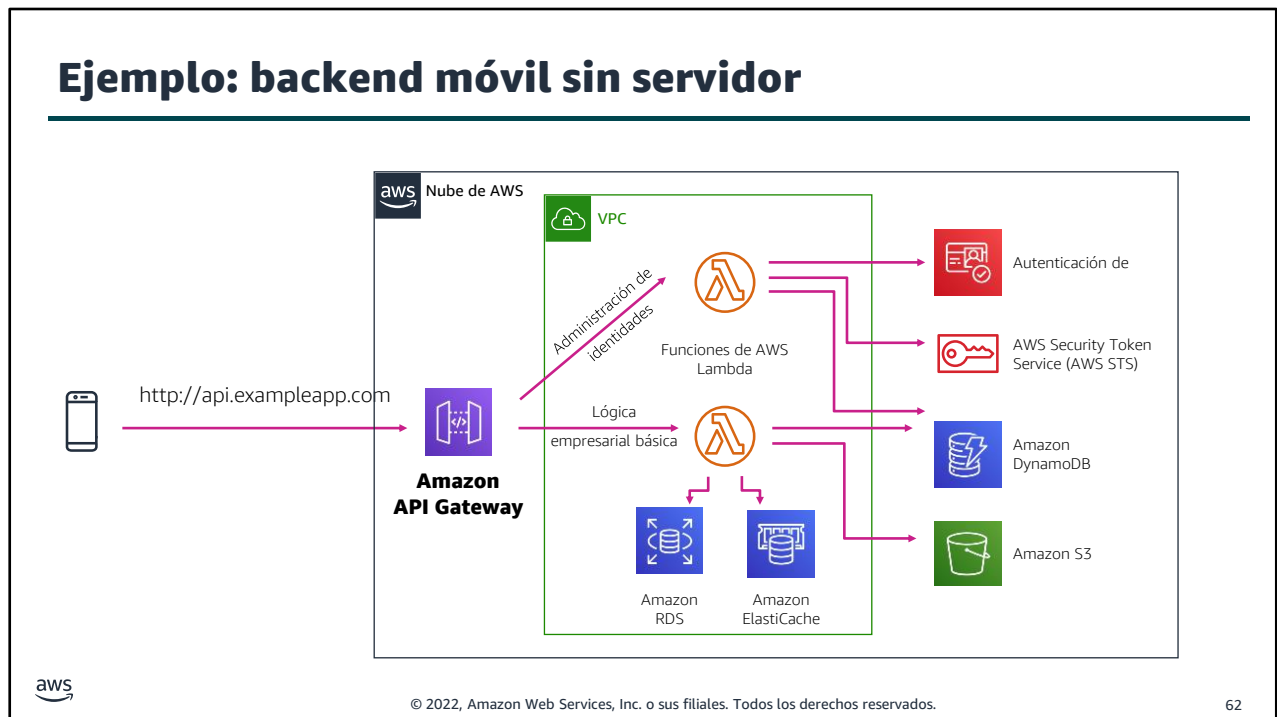
© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

61

Amazon API Gateway está profundamente integrado con AWS Lambda. Este ejemplo muestra cómo los dos servicios pueden utilizarse juntos en una arquitectura para una aplicación de microservicios RESTful.

En este ejemplo, los clientes pueden utilizar su microservicio realizando llamadas HTTP API. Amazon API Gateway aloja solicitudes RESTful HTTP de sus clientes y las respuestas a estas. En este escenario, API Gateway proporciona autorización integrada, limitación controlada, seguridad, tolerancia a errores, mapeo de solicitud-respuesta y optimización del rendimiento. AWS Lambda contiene la lógica empresarial para procesar las llamadas entrantes a la API y utiliza DynamoDB como almacenamiento persistente. Amazon DynamoDB almacena de forma persistente los datos de los microservicios y escala en función de la demanda. Dado que los microservicios están diseñados para hacer una cosa de forma eficiente, se suele incorporar un almacén de datos NoSQL sin esquema.

Ejemplo: backend móvil sin servidor



Analicemos otro ejemplo de cómo se puede utilizar Amazon API Gateway con Lambda en una arquitectura sin servidor como puerta de enlace al *backend* para una aplicación móvil. Se espera que las aplicaciones móviles ofrezcan al usuario una experiencia rápida, consistente y rica en funciones, además de que suelen tener una presencia mundial. Además, los patrones de los usuarios móviles son dinámicos, con picos de uso impredecibles. Las aplicaciones móviles requieren un amplio conjunto de servicios móviles que funcionen juntos a la perfección sin sacrificar el control ni la flexibilidad de la infraestructura de *backend*.

En este ejemplo, una aplicación móvil envía una solicitud a Amazon API Gateway, que reenvía la solicitud a una función de Lambda que llama a Amazon Cognito y AWS Security Token Service (AWS STS) para administrar la identidad. Amazon Cognito autentica a los usuarios de su aplicación móvil mediante proveedores de identidad (IdP) externos compatibles con lenguaje de marcado para confirmaciones de seguridad (SAML) u OpenID Connect, IdP sociales (por ejemplo, Facebook, Twitter, Amazon) e IdP personalizados. Después de confirmar la identidad del usuario móvil, otra función de Lambda ejecuta la lógica empresarial principal de la aplicación.

Conclusiones importantes de la Sección 6



- **Amazon API Gateway** es un servicio completamente administrado que le permite **crear, publicar, mantener, supervisar y proteger API** a cualquier escala.
- Amazon API Gateway funciona como **punto de entrada a los recursos de backend para sus aplicaciones**. Resume y expone las API que pueden llamar a varias aplicaciones de **backend**. Estas aplicaciones incluyen funciones de Lambda, contenedores de Docker que se ejecutan en instancias de EC2, VPC o cualquier punto de enlace accesible de forma pública.
- Amazon API Gateway está **profundamente integrado con Lambda**.

© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

63

Entre los puntos clave de esta sección del módulo, se incluyen los siguientes:

- Amazon API Gateway es un servicio completamente administrado que le permite crear, publicar, mantener, supervisar y proteger API a cualquier escala.
- Amazon API Gateway funciona como punto de entrada a los recursos de *backend* para sus aplicaciones. Resume y expone las API que pueden llamar a varias aplicaciones de *backend*. Estas aplicaciones incluyen funciones de Lambda, contenedores de Docker que se ejecutan en instancias de EC2, VPC o cualquier punto de enlace accesible de forma pública.
- Amazon API Gateway está profundamente integrado con Lambda.

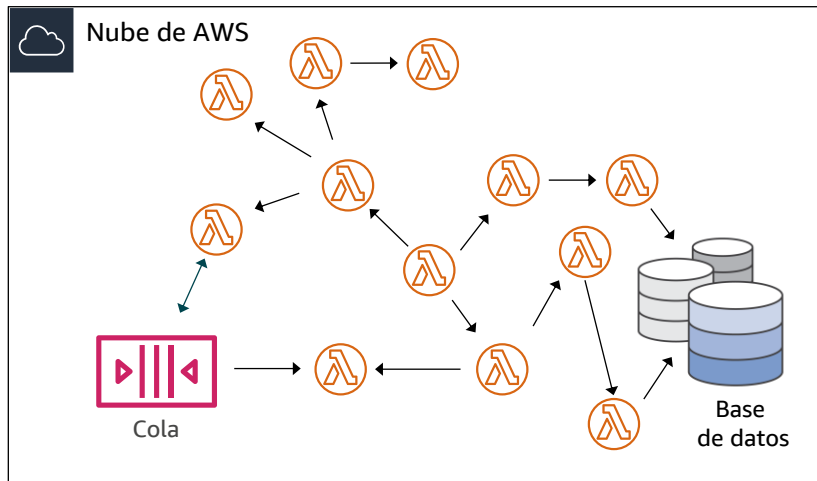
Sección 7: Organización de microservicios con AWS Step Functions

Módulo 13: Creación de microservicios y arquitecturas sin servidor



Introducción a la sección 7: Organización de microservicios con AWS Step Functions.

Desafíos de las aplicaciones de microservicios



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

65

A medida que la aplicación crece en tamaño, el número de componentes aumenta y surgen muchos patrones posibles de qué tareas ejecutar y en qué orden. Por ejemplo, consideremos una aplicación de microservicio que se crea utilizando funciones de Lambda. Es posible que desee invocar una función de Lambda inmediatamente después de otra función, y solo si la primera función se ejecuta correctamente. Es posible que desee invocar dos funciones en paralelo y, a continuación, enviar los resultados combinados a una tercera función. O puede que desee elegir cuál de las dos funciones se invoca basándose en la salida de otra función.

La invocación de una función puede derivar en un error por varias razones. El código puede plantear una excepción, agotar el tiempo de espera o quedarse sin memoria. El tiempo de ejecución que ejecuta el código puede encontrar un error y detenerse. Cuando se produce un error, es posible que su código se haya ejecutado completamente, parcialmente o no se haya ejecutado. En la mayoría de los casos, el cliente o servicio que invoca la función reintentará la ejecución si encuentra un error, por lo que el código debe tener la capacidad de procesar el mismo evento repetidamente sin que surjan efectos no deseados. Si su función administra recursos o escribe en una base de datos, debe manejar los casos en los que la misma solicitud se realiza varias veces.

Necesita una forma de coordinar los componentes de su aplicación. Esta capa de coordinación se debe poder escalar automáticamente en función de los cambios en las cargas de trabajo y administrar los errores y los tiempos de espera. También debe mantener el estado de su aplicación mientras se está ejecutando, como el seguimiento de los pasos actuales y el almacenamiento de los datos que se mueven entre los pasos del flujo de trabajo. Estas funciones le ayudarán a crear y utilizar sus aplicaciones. También desea tener visibilidad de su aplicación para poder solucionar errores y controlar el rendimiento.

AWS Step Functions



- Coordina los microservicios mediante flujos de trabajo visuales
- Permite hacer un recorrido por las funciones de su aplicación
- Desencadena automáticamente y realiza un seguimiento de cada paso
- Proporciona captura y registro de errores simples si falla un paso

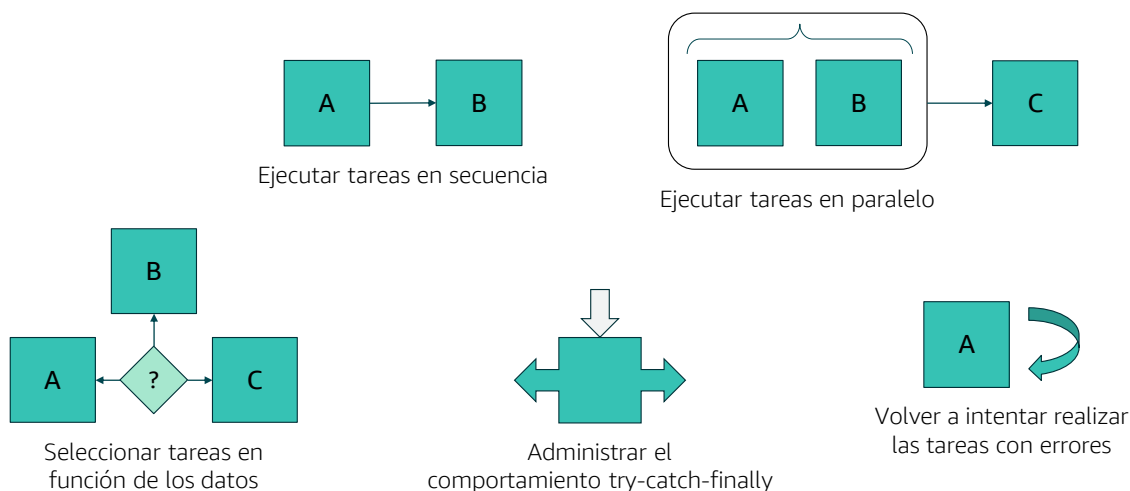


© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

66

[AWS Step Functions](#) es un servicio web que facilita la coordinación de componentes de aplicaciones y microservicios distribuidos mediante flujos de trabajo visuales. Step Functions proporciona una manera fiable de coordinar los componentes y procesar las funciones de su aplicación. Step Functions ofrece una consola gráfica para que pueda visualizar los componentes de su aplicación en varios pasos. Activa cada paso y le da seguimiento de manera automática, y realiza reintentos cuando se producen errores, por lo que su aplicación se ejecuta en orden y según lo previsto. Step Functions registra el estado de cada paso, de manera puede diagnosticar y depurar los problemas con rapidez.

Coordinación del flujo de trabajo



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

67

AWS Step Functions administra la lógica de su aplicación por usted. Implementa primitivas básicas, como ejecutar tareas de forma secuencial o en paralelo, ramificar y establecer tiempos de espera. Esta técnica elimina el código adicional que puede repetirse en los microservicios y funciones. AWS Step Functions administra automáticamente los errores y las excepciones gracias a la función integrada “intentar, capturar y reintentar”, ya sea que la tarea tarde segundo o meses en completarse. Puede reintentar ejecutar automáticamente las tareas que presentaron errores o interrupciones. Puede responder de otra forma a los distintos tipos de errores y recuperarse adecuadamente recurriendo a códigos de limpieza y recuperación designados.

Máquinas de estados



Una **máquina de estados** es una colección de estados que pueden realizar un trabajo.

Máquina expendedora

En espera de la transacción



Selección de refrescos



Venta de refrescos



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

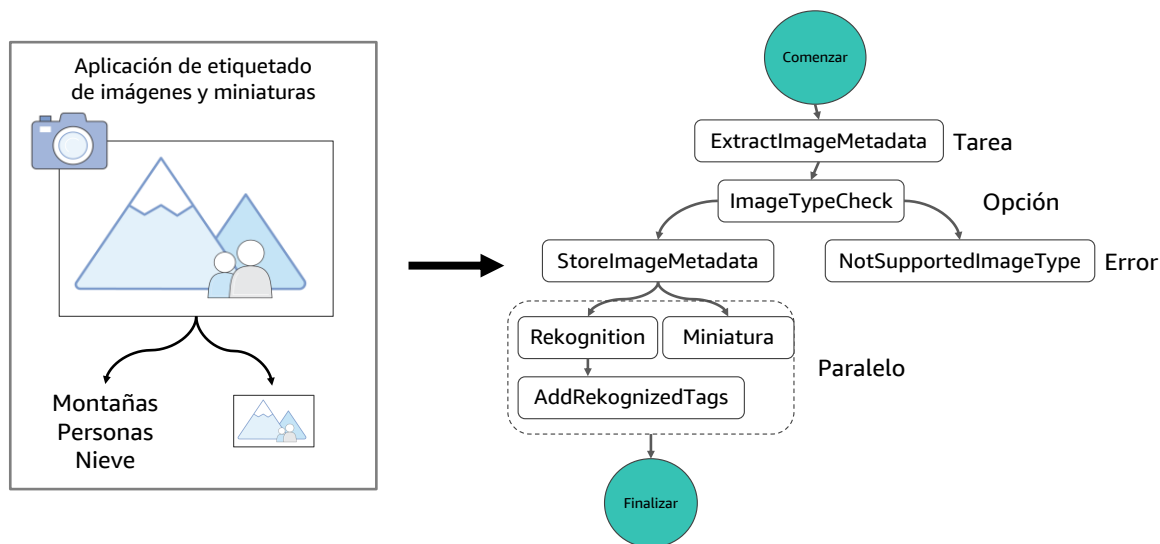
68

AWS Step Functions le permite crear y automatizar sus propias máquinas de estado dentro del entorno de AWS mediante el uso de Amazon States Language basado en JSON. Este lenguaje contiene una estructura compuesta por varios estados, tareas, elecciones, gestión de errores y más.

Una *máquina de estados* es una colección de estados que pueden realizar un trabajo.

Un ejemplo común de una máquina de estados es una máquina expendedora de refrescos. La máquina se inicia en estado operativo (a la espera de una transacción) y, luego, pasa a la selección de refrescos cuando se le agrega dinero. A continuación, pasa al estado Venta, en el que el refresco se entrega al cliente. Después de terminar la transacción, el estado vuelve al estado operativo.

Estados



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

69

Una máquina de estados finitos puede expresar un algoritmo como un número de estados, sus relaciones y su entrada y salida. Los *estados* son elementos de la máquina de estados. Los estados individuales pueden tomar decisiones en función de la entrada, realizar acciones y transferir la salida a otros estados.

Tipos de estado

Tarea	Una sola unidad de trabajo realizada por una máquina de estados
Opción	Incorpora lógica de ramificación a una máquina de estados
Error	Detiene una máquina de estados en ejecución y la marca como error
Correcto	Detiene correctamente una máquina de estados en ejecución
Aprobado	Paso de entrada a la salida, sin realizar ningún trabajo
Esperar	Retrasa la continuación durante un tiempo determinado
Paralelo	Crea ramificaciones de ejecución paralelas en su máquina de estados
Mapa	Itera los pasos de forma dinámica



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

70

Los estados pueden realizar varias funciones en una máquina de estado:

- Realizar algunas tareas en la máquina de estado (un estado *Task* [Tarea])
- Elegir entre las ramificaciones de las máquinas de estado que se van a ejecutar (un estado *Choice* [Elección])
- Detener una máquina de estados en ejecución con un error o que se realizó correctamente (un estado *Fail* [Error] o *Succeed* [Éxito])
- Pasar los datos de entrada a la salida o insertar ciertos datos fijos (un estado *Pass* [Aprobado])
- Otorgar un plazo que dure cierto tiempo o hasta una fecha y hora determinada (un estado *Wait* [Espera])
- Comenzar ramificaciones de ejecución paralelas en su máquina de estados (un estado *Parallel* [Paralelo])
- Iterar pasos de forma dinámica (un estado *Map* [Mapeo])

Para obtener más información sobre los tipos de estado, consulte [Estados](#) en la documentación de AWS.

Amazon States Language

Define el flujo de trabajo en JSON mediante Amazon States Language

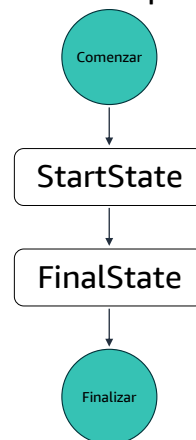
```
{
  "Comment": "An example of the ASL.",
  "StartAt": "StartState",
  "States": {
    "StartState": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east...",
      "Next": "FinalState"
    }
    "FinalState": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east...",
      "End": true
    }
  }
}
```



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

71

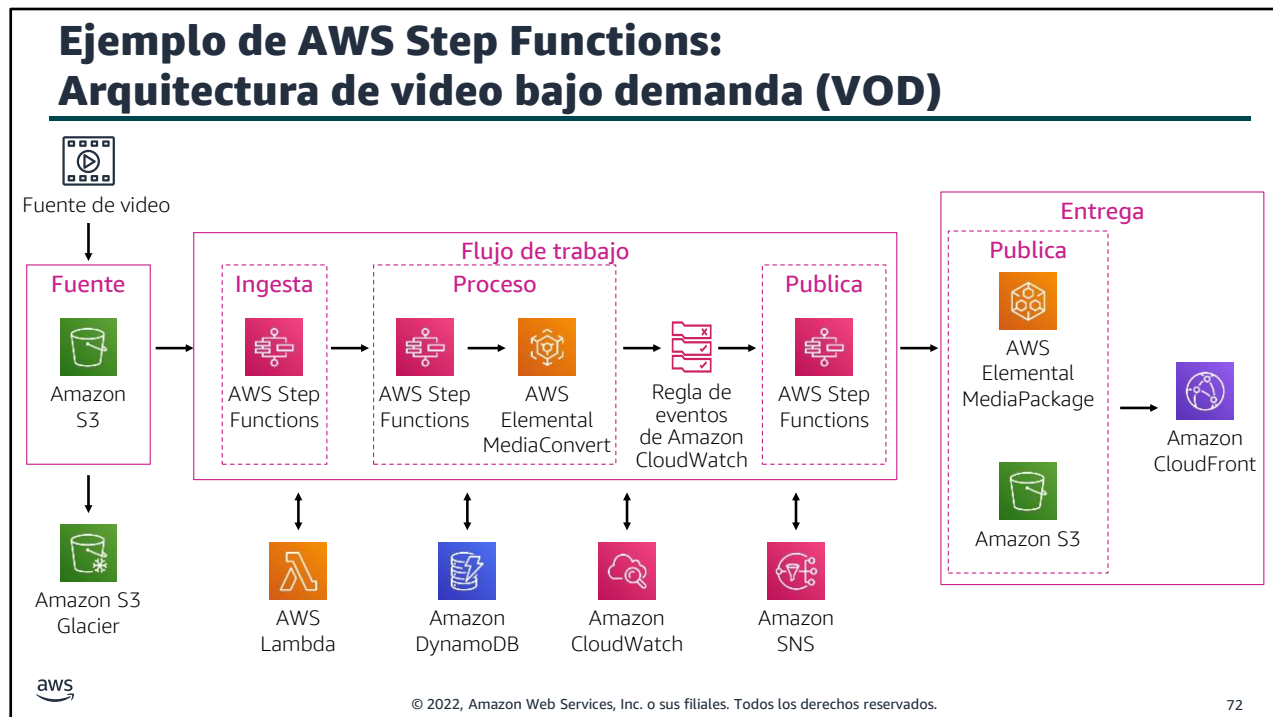
Visualiza el flujo de trabajo en la consola Step Functions



Las máquinas de estado se definen utilizando Amazon States Language. Amazon States Language es un lenguaje estructurado basado en JSON. A continuación, AWS Step Functions representa la estructura JSON en una vista gráfica en tiempo real. De este modo, puede visualizar su máquina de estados directamente en la consola de Step Functions.

Este es un ejemplo de una máquina de estados con dos tipos de estado de tarea.

Para obtener más información sobre Amazon States Language, consulte la [documentación de AWS](#).



Este diagrama de arquitectura muestra un ejemplo de caso práctico de AWS Step Functions en una solución de video bajo demanda (VOD). Otro componente clave de esta arquitectura es AWS Elemental MediaConvert, un servicio de transcodificación de video basado en archivos con funciones de transmisión. Permite crear contenido VOD para la transmisión y distribución en pantallas múltiples a gran escala.

En esta solución, los videos de origen y los archivos de metadatos se ingestan y procesan para su reproducción en una amplia gama de dispositivos.

- AWS Step Functions crea Step Functions de **Ingest** (Ingerir), **Process** (Procesar) y **Publish** (Publicar).
- Las funciones de AWS Lambda realizan el trabajo de cada paso y procesan los mensajes de error.
- Los buckets de Amazon S3 almacenan archivos multimedia de origen y destino.
- Amazon CloudWatch se utiliza para el registro.
- Reglas de eventos de Amazon CloudWatch para notificaciones de AWS Elemental MediaConvert.
- Una tabla de Amazon DynamoDB almacena los datos capturados mediante el flujo de trabajo.
- Los temas de Amazon SNS envían notificaciones de codificación, publicación y error.
- Los archivos multimedia transcodificados se almacenan para entregarse a los usuarios bajo demanda a través de Amazon CloudFront.

Para más información sobre esta arquitectura, consulte [Información general sobre la arquitectura](#).

Conclusiones importantes de la Sección 7



- **AWS Step Functions** es un servicio web que facilita la coordinación de componentes de aplicaciones y microservicios distribuidos mediante flujos de trabajo visuales
- AWS Step Functions le permite **crear y automatizar sus propias máquinas de estado** dentro del entorno de AWS
- AWS Step Functions **administra la lógica de su aplicación e implementa primitivas básicas**, como las ramificaciones de ejecución paralelas o secuenciales, y los tiempos de espera
- Las máquinas de estado se definen utilizando **Amazon States Language**

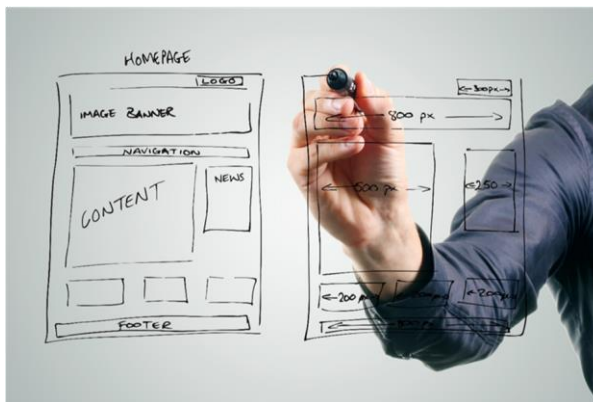
© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

73

Entre los puntos clave de esta sección del módulo, se incluyen los siguientes:

- AWS Step Functions es un servicio web que facilita la coordinación de componentes de aplicaciones y microservicios distribuidos mediante flujos de trabajo visuales
- AWS Step Functions le permite crear y automatizar sus propias máquinas de estado dentro del entorno de AWS
- AWS Step Functions administra la lógica de su aplicación e implementa primitivas básicas, como las ramificaciones de ejecución paralelas o secuenciales, y los tiempos de espera
- Las máquinas de estado se definen utilizando Amazon States Language

Módulo 13. Laboratorio de desafíos: Implementación de una arquitectura sin servidor para la cafetería



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

74

Ahora completará el módulo 13. Laboratorio de desafíos: Implementación de una arquitectura sin servidor para la cafetería.

La necesidad empresarial: Un entorno sin servidores



Frank y Martha quieren recibir informes diarios por correo electrónico sobre todos los pedidos que se realizaron en el sitio web. Olivia le recomienda a Sofía y a Nikhil separar los informes de tareas que no sean críticas para la empresa de la instancia de servidor web de producción.



Sofía y Nikhil quieren desacoplar más la arquitectura y trasladar el trabajo cron a un entorno administrado y sin servidor, que escale bien y reduzca los costos.



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

75

Frank y Martha quieren recibir informes diarios por correo electrónico sobre todos los pedidos realizados en el sitio web. Frank quiere anticiparse a la demanda y hornear el número correcto de postres en el futuro (reduciendo el desperdicio). Martha quiere identificar patrones en su empresa (análisis). Actualmente, Sofía ha configurado un trabajo cron en la instancia del servidor web que envía estos mensajes de correo electrónico con el informe diario de los pedidos a Frank y Martha. Sin embargo, el trabajo cron consume muchos recursos y reduce el rendimiento del servidor web.

Olivia aconseja a Sofía y a Nikhil separar los informes de las tareas que no sean críticas para la empresa. Sofía y Nikhil quieren desacoplar más la arquitectura y trasladar el trabajo cron a un entorno administrado y sin servidor, que escale bien y reduzca los costos.

Laboratorio de desafíos: tareas

1. Descargar el código fuente
2. Crear la función de Lambda *DataExtractor* en la VPC
3. Crear la función de Lambda *salesAnalysisReport*
4. Crear un tema de SNS
5. Crear una suscripción por correo al tema de SNS
6. Probar la función de Lambda *salesAnalysisReport*
7. Configurar un evento de Amazon EventBridge para desencadenar la función de Lambda todos los días



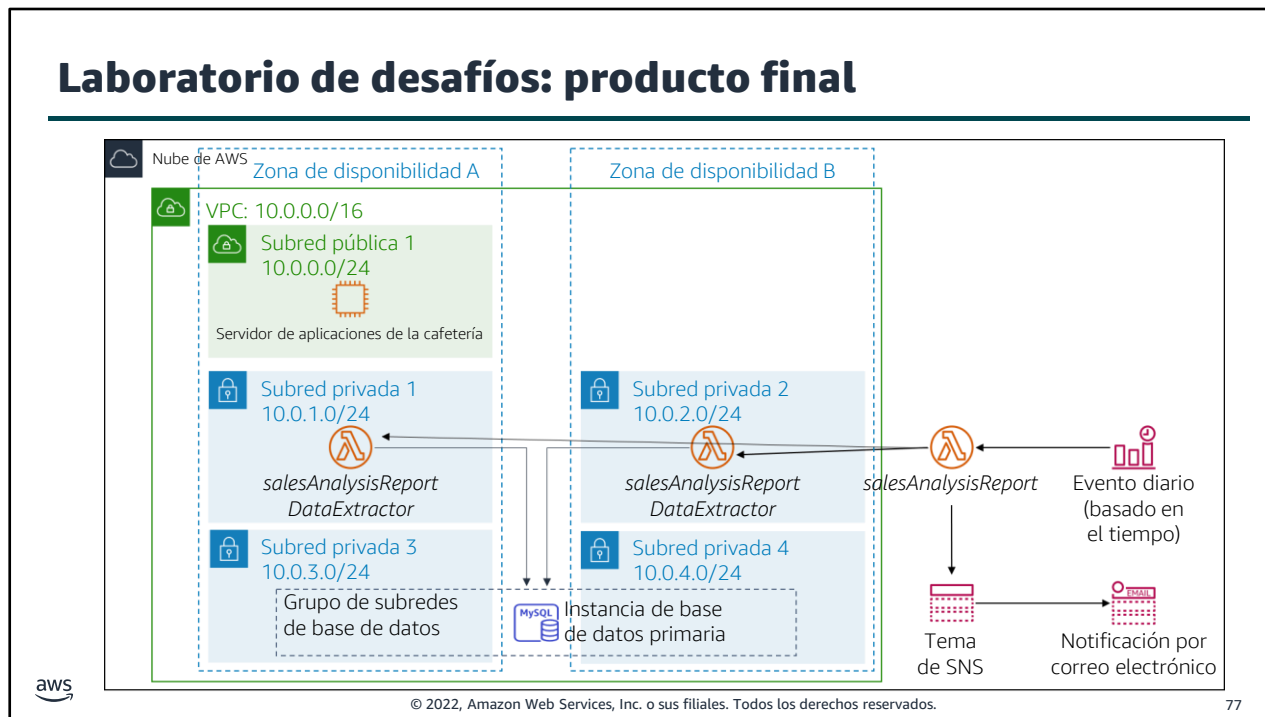
© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

76

En este laboratorio de desafíos, realizará las siguientes tareas:

1. Descargar el código fuente
2. Crear la función de Lambda *DataExtractor* en la VPC
3. Crear la función de Lambda *salesAnalysisReport*
4. Crear un tema de SNS
5. Crear una suscripción por correo al tema de SNS
6. Probar la función de Lambda *salesAnalysisReport*
7. Configurar un evento de Amazon EventBridge para desencadenar la función de Lambda todos los días

Laboratorio de desafíos: producto final



El diagrama resume lo que creará después de completar el laboratorio.

Para mejorar la accesibilidad: diagrama de una arquitectura de tres niveles con un servidor de aplicaciones de la cafetería en la subred pública 1, una función de Lambda del extractor de datos para los informes de análisis de ventas en las subredes privadas 1 y 2, y una instancia de base de datos primaria en un grupo de subredes de bases de datos que abarca las subredes privadas 3 y 4. El evento diario desencadena la función de Lambda de informe de análisis de ventas, que desencadena la función de Lambda del extractor de datos, y también una notificación por correo electrónico que se envía desde el tema de SNS. **Fin de la descripción de accesibilidad.**



~ 90 minutos



Inicio del módulo 13: Laboratorio de desafíos: Implementación de una arquitectura sin servidor para la cafetería



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

78

Es momento de comenzar con el laboratorio de desafíos.

Análisis posterior del laboratorio de desafíos Aprendizajes clave



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

79

Después de completar el laboratorio de desafíos, su instructor puede optar por dirigir una conversación sobre los aprendizajes clave del laboratorio.

Conclusión del módulo

Módulo 13: Creación de microservicios y arquitecturas sin servidor



Ahora es el momento de revisar el módulo y concluir con una evaluación de conocimientos y una discusión sobre una pregunta del examen de certificación de práctica.

Resumen del módulo

En resumen, en este módulo aprendieron a hacer lo siguiente:

- Indicar las características de los microservicios
- Refactorizar una aplicación monolítica en microservicios y utilizar Amazon ECS para implementar los microservicios en contenedores
- Explicar la arquitectura sin servidor
- Implementar una arquitectura sin servidor con AWS Lambda
- Describir una arquitectura común para Amazon API Gateway
- Describir los tipos de flujos de trabajo que admite AWS Step Functions



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

81

En resumen, en este módulo aprendieron a hacer lo siguiente:

- Indicar las características de los microservicios
- Refactorizar una aplicación monolítica en microservicios y utilizar Amazon ECS para implementar los microservicios en contenedores
- Explicar la arquitectura sin servidor
- Implementar una arquitectura sin servidor con AWS Lambda
- Describir una arquitectura común para Amazon API Gateway
- Describir los tipos de flujos de trabajo que admite AWS Step Functions

Completar la evaluación de conocimientos



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

82

Ahora es el momento de completar la evaluación de conocimientos para este módulo.

Pregunta de examen de ejemplo



Una organización aloja 10 microservicios, cada uno en un grupo de Auto Scaling detrás de los equilibradores de carga clásicos. Cada instancia de EC2 se ejecuta con una carga óptima.

¿Cuál de las siguientes acciones permitiría a la organización reducir los costos sin afectar al rendimiento?

Opción	Respuesta
A	Reducir el número de instancias de EC2 detrás de cada equilibrador de carga clásico.
B	Cambiar los tipos de instancia en la configuración de lanzamiento del grupo de Auto Scaling.
C	Cambiar el tamaño máximo, pero dejar la capacidad deseada de los grupos de Auto Scaling.
D	Reemplazar los equilibradores de carga clásicos por un solo equilibrador de carga de aplicación.

© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

83

Mire las opciones de respuesta y descártelas según las palabras clave.

Respuesta a la pregunta de examen de ejemplo



Una organización aloja 10 microservicios, cada uno en un grupo de Auto Scaling detrás de los equilibradores de carga clásicos. Cada instancia de EC2 se ejecuta con una carga óptima.

¿Cuál de las siguientes acciones permitiría a la organización reducir los costos sin afectar al rendimiento?

La respuesta correcta es la opción D.

Las palabras clave de la pregunta son reducir costos sin afectar al rendimiento y reemplazar los equilibradores de carga clásicos por un solo equilibrador de carga de aplicaciones.

© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

84

Las siguientes palabras clave que hay que reconocer: reducir costos sin afectar al rendimiento y reemplazar los equilibradores de carga clásicos por un solo equilibrador de carga de aplicaciones.

La respuesta correcta es D: “Reemplazar los equilibradores de carga clásicos por un solo equilibrador de carga de aplicación”. Por eliminación, la opción D es correcta. También puede utilizar un solo equilibrador de carga de aplicación para enrutar las solicitudes a todos los servicios de su aplicación en lugar de utilizar un solo equilibrador de carga clásico para cada microservicio.

Respuestas incorrectas:

- La opción A se puede eliminar: porque las instancias de EC2 se están ejecutando con una carga óptima y se sobrecargarán si reduce su número.
- La opción B se puede eliminar: no reducirá los costos si elige un tamaño de instancia mayor. Por otro lado, si disminuye el tamaño de la instancia, las instancias de EC2 se sobrecargarán porque ya están funcionando con una carga óptima.
- La opción C también se puede eliminar: los costos serán los mismos o aumentarán si se incrementa el tamaño máximo y se mantiene la capacidad deseada.

Recursos adicionales

- Proyecto: [Dividir una aplicación monolítica en microservicios](#)
- Documento técnico sobre [Arquitecturas sin servidor con AWS Lambda](#)
- Documento técnico sobre [Arquitecturas de varios niveles sin servidor de AWS con Amazon API Gateway y AWS Lambda](#)
- Documento técnico sobre el [Marco de AWS Well-Architected: Modo de aplicación sin servidor](#)
- Tutorial: [Creación y uso de funciones de Lambda](#)



© 2022, Amazon Web Services, Inc. o sus filiales. Todos los derechos reservados.

85

Si desea obtener más información sobre los temas tratados en este módulo, es posible que le resulten útiles los siguientes recursos adicionales:

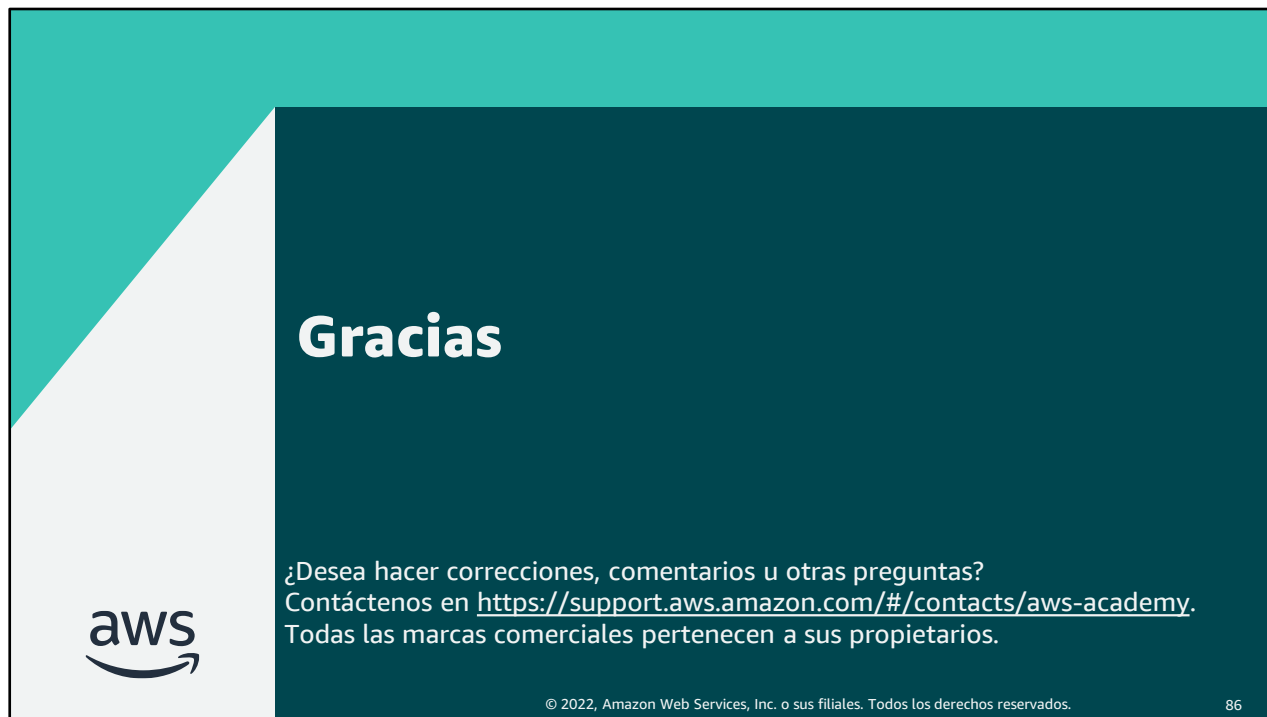
Proyecto: [Dividir una aplicación monolítica en microservicios](#)

Documento técnico sobre [Arquitecturas sin servidor con AWS Lambda](#)

Documento técnico sobre [Arquitecturas de varios niveles sin servidor de AWS con Amazon API Gateway y AWS Lambda](#)

Documento técnico sobre el [Marco de AWS Well-Architected: Modo de aplicación sin servidor](#)

Tutorial: [Creación y uso de funciones de Lambda](#)



Gracias por completar este módulo.